

恶意代码分析与防治技术实验报告

Lab 12



网络空间安全学院

信息安全专业

2211985 李佳璐

一、实验目的

完成课本Lab12的实验内容，编写Yara规则，并尝试IDA Python的自动化分析

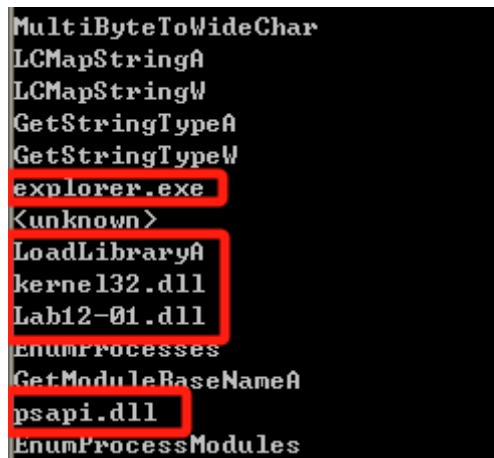
二、实验过程

Lab12-01

1.过程分析

- 静态分析

使用strings工具查看字符串列表



```
MultiByteToWideChar
LCMaPStringA
LCMaPStringW
GetStringTypeA
GetStringTypeW
explorer.exe
<unknown>
LoadLibraryA
kernel32.dll
Lab12-01.dll
EnumProcesses
GetModuleBaseNameA
psapi.dll
EnumProcessModules
```

可以看到 explorer.exe、Lab12-01.dll 以及 psapi.dll 等，推测是恶意代码的一些目标文件。同时可以看到 Loadlibrary 等函数，推测恶意代码会使用这些函数进行攻击。

查看其导入函数：

Address	Ordinal	Name	Library
000000...		CloseHandle	KERNEL32
000000...		OpenProcess	KERNEL32
000000...		CreateRemoteThread	KERNEL32
000000...		GetModuleHandleA	KERNEL32
000000...		WriteProcessMemory	KERNEL32
000000...		VirtualAllocEx	KERNEL32
000000...		lstrcatA	KERNEL32
000000...		GetCurrentDirectoryA	KERNEL32
000000...		GetProcAddress	KERNEL32
000000...		LoadLibraryA	KERNEL32
000000...		GetCommandLineA	KERNEL32
000000...		GetVersion	KERNEL32
000000...		ExitProcess	KERNEL32
000000...		TerminateProcess	KERNEL32
000000...		GetCurrentProcess	KERNEL32
000000...		UnhandledExceptionFilter	KERNEL32
000000...		GetModuleFileNameA	KERNEL32
000000...		FreeEnvironmentStringsA	KERNEL32
000000...		FreeEnvironmentStringsW	KERNEL32
000000...		WideCharToMultiByte	KERNEL32
000000...		GetEnvironmentStrings	KERNEL32
000000...		GetEnvironmentStringsW	KERNEL32
000000...		SetHandleCount	KERNEL32
000000...		GetStdHandle	KERNEL32
000000...		GetFileType	KERNEL32
000000...		GetStartupInfoA	KERNEL32
000000...		GetEnvironmentVariableA	KERNEL32
000000...		GetVersionExA	KERNEL32
000000...		HeapDestroy	KERNEL32
000000...		HeapCreate	KERNEL32
000000...		VirtualFree	KERNEL32
000000...		HeapFree	KERNEL32
000000...		RtlUnwind	KERNEL32

检查Lab12-01.exe的导入函数，我们看见 CreateRemoteThread、WriteProcessMemory 以及 VirtualAllocEx。推测可能是某种形式的进程注入行为。

• 动态分析

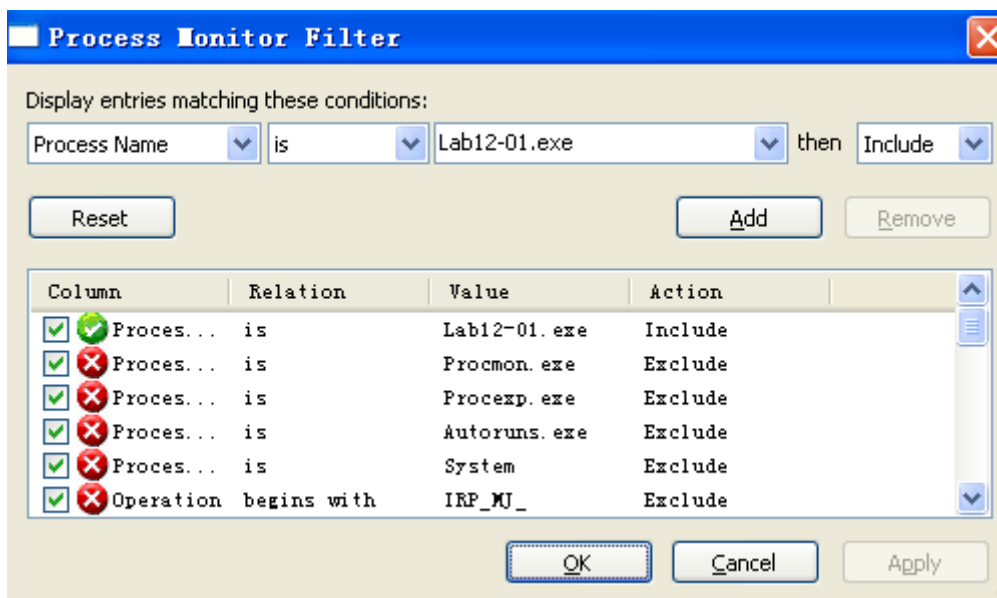
运行Lab12-01.exe，可以看到一个创建了消息框：



它会每分钟会创建一个消息框（消息框编号递增）



使用Procomon监视恶意代码运行，设置过滤器：



可以看到对 psapi.dll 的操作:

19:3...	Lab12-01.exe	1844	CreateFile	C:\WINDOWS\system32\sortkey.nls	SUCCESS	Desired Acces...
19:3...	Lab12-01.exe	1844	CreateFileM...	C:\WINDOWS\system32\sortkey.nls	SUCCESS	SyncType: Syn...
19:3...	Lab12-01.exe	1844	QueryStanda...	C:\WINDOWS\system32\sortkey.nls	SUCCESS	AllocationSiz...
19:3...	Lab12-01.exe	1844	CreateFileM...	C:\WINDOWS\system32\sortkey.nls	SUCCESS	SyncType: Syn...
19:3...	Lab12-01.exe	1844	CreateFile	C:\WINDOWS\system32\psapi.dll	SUCCESS	Desired Acces...
19:3...	Lab12-01.exe	1844	CreateFileM...	C:\WINDOWS\system32\psapi.dll	SUCCESS	SyncType: Syn...
19:3...	Lab12-01.exe	1844	QueryStanda...	C:\WINDOWS\system32\psapi.dll	SUCCESS	AllocationSiz...
19:3...	Lab12-01.exe	1844	CreateFileM...	C:\WINDOWS\system32\psapi.dll	SUCCESS	SyncType: Syn...
19:3...	Lab12-01.exe	1844	CloseFile	C:\WINDOWS\system32\ntdll.dll	SUCCESS	

使用IDA Pro深入分析:

在main函数开始处的几行, 可以看到这个恶意代码在解析 psapi.dll 中Windows枚举进程的函数:

```

mov     [ebp+var_118], 0
push    offset ProcName ; "EnumProcessModules"
push    offset LibFileName ; "psapi.dll"
call    ds:LoadLibraryA
push    eax              ; hModule
call    ds:GetProcAddress
mov     dword_408714, eax
push    offset aGetmodulebasen ; "GetModuleBaseNameA"
push    offset LibFileName ; "psapi.dll"
call    ds:LoadLibraryA
push    eax              ; hModule
call    ds:GetProcAddress
mov     dword_40870C, eax
push    offset aEnumprocesses ; "EnumProcesses"
push    offset LibFileName ; "psapi.dll"
call    ds:LoadLibraryA
push    eax              ; hModule
call    ds:GetProcAddress
mov     dword_408710, eax
lea     ecx, [ebp+Buffer]
push    ecx              ; lpBuffer
push    104h             ; nBufferLength
call    ds:GetCurrentDirectoryA
push    offset String2 ; ""
lea     edx, [ebp+Buffer]
push    edx              ; lpString1
call    ds:lstrcatA
push    offset aLab1201_dll ; "Lab12-01.dll"
lea     eax, [ebp+Buffer]
push    eax              ; lpString1
call    ds:lstrcatA

```

恶意代码使用了 LoadLibraryA 和 GetProcAddress 手动地解析3个函数

这个恶意代码保存指向 `dword_408714`、`dword_40870C` 以及 `dword_408710` 的函数指针

在动态解析这些函数之后，这段代码调用 `dword_408710`，它获取在系统中每一个进程对象的PID：

```
push    ecx
push    1000h
lea     edx, [ebp+dwProcessId]
push    edx
call    dword_408710
test    eax, eax
jnz     short loc_4011D0
```

返回一个由局部变量 `dwProcessId` 引用的PID数组，`dwProcessId` 被用来在一个循环中迭代进程列表，并对每一个PID调用 `sub_401000`。

检查 `sub_401800` 时，我们看到这个动态解析的导入函数在PID被传给 `OpenProcess` 函数后被调用。接下来，我们在处看到一个对 `dword_40878C` 的调用。

```
.text:00401078      push    104h
.text:0040107D      lea     ecx, [ebp+var_108]
.text:00401083      push    ecx
.text:00401084      mov     edx, [ebp+var_10C]
.text:0040108A      push    edx
.text:0040108B      mov     eax, [ebp+hObject]
.text:0040108E      push    eax
.text:0040108F      call    dword_40870C
.text:00401095      loc_401095:
.text:00401095      ; CODE XREF: sub_401000+58↑j
.text:00401095      ; sub_401000+76↑j
.text:00401095      push    0Ch
.text:00401095      ; size_t
.text:00401097      push    offset aExplorer_exe ; "explorer.exe"
.text:0040109C      lea     ecx, [ebp+var_108]
.text:004010A2      push    ecx
.text:004010A3      call    strnicmp
```

这个动态解析的函数 `dword_40870C` 被用来将PID翻译为进程名。在这个调用后，看到通过 `dword_40870C` 获得的字符串(str1)与 `explorer.exe` (str2)之间的比较可以看出，这个恶意代码在内存中查找 `explorer.exe` 进程。

一旦 `explorer.exe` 被找到，`sub_401000` 函数将返回1，并且这个main函数将调用 `OpenProcess`，来打开一个指向它的句柄。

如果这个恶意代码成功地获取这个进程的句柄，将会执行以下代码：

```
.text:0040128C loc_40128C:
.text:0040128C      push    4
.text:0040128E      push    3000h
.text:00401293      push    104h
.text:00401298      push    0
.text:0040129A      mov     edx, [ebp+hProcess]
.text:004012A0      push    edx
.text:004012A1      call    ds:VirtualAllocEx
.text:004012A7      mov     [ebp+lpBaseAddress], eax
.text:004012AD      cmp     [ebp+lpBaseAddress], 0
.text:004012B4      jnz     short loc_4012BE
.text:004012B6      or      eax, 0FFFFFFFh
.text:004012B9      jmp     loc_401342
```

并且这个句柄 `hProcess` 将被用来操纵这个进程。

上面这段代码可以看到对 `VirtualAllocEx` 的一个调用。这动态地在 `explorer.exe` 中分配内存：`0x104`字节压入 `dwSize` 而被分配。

```

loc_40128C:                ; flProtect
push    4
push    3000h               ; flAllocationType
push    104h               ; dwSize
push    0                  ; lpAddress
mov     edx, [ebp+hProcess]
push    edx                ; hProcess
call    ds:VirtualAllocEx
mov     [ebp+lpBaseAddress], eax
cmp     [ebp+lpBaseAddress], 0
jnz     short loc_4012BE

```

```

loc_4012BE:                ; lpNumberOfBytesWritten
push    0
push    104h               ; nSize
lea     eax, [ebp+Buffer]
push    eax                ; lpBuffer
mov     ecx, [ebp+lpBaseAddress]
push    ecx                ; lpBaseAddress
mov     edx, [ebp+hProcess]
push    edx                ; hProcess
call    ds:WriteProcessMemory
push    offset ModuleName, "kernel32.dll"
call    ds:GetModuleHandleA
mov     [ebp+hModule], eax
push    offset aLoadlibrarya ; "LoadLibraryA"
mov     eax, [ebp+hModule]
push    eax                ; hModule
call    ds:GetProcAddress
mov     [ebp+lpStartAddress], eax

```

如果 `VirtualAllocEx` 成功，一个指向被分配内存的指针将被移动到 `lpBaseAddress` 中，并和进程句柄一起传给 `writeProcessMemory`，以便向 `explorer.exe` 写入 `lpBuffer` 中的数据。

跟踪代码回溯到 `lpBuffer` 被设置的地方：

```

mov     [ebp+var_118], 0
push    offset ProcName ; "EnumProcessModules"
push    offset LibFileName ; "psapi.dll"
call    ds:LoadLibraryA
push    eax                ; hModule
call    ds:GetProcAddress
mov     dword_408714, eax
push    offset aGetModulebasen ; "GetModuleBaseNameA"
push    offset LibFileName ; "psapi.dll"
call    ds:LoadLibraryA
push    eax                ; hModule
call    ds:GetProcAddress
mov     dword_40870C, eax
push    offset aEnumprocesses ; "EnumProcesses"
push    offset LibFileName ; "psapi.dll"
call    ds:LoadLibraryA
push    eax                ; hModule
call    ds:GetProcAddress
mov     dword_408710, eax
lea     ecx, [ebp+Buffer]
push    ecx                ; lpBuffer
push    104h                ; nBufferLength
call    ds:GetCurrentDirectoryA
push    offset String2 ; ""
lea     edx, [ebp+Buffer]
push    edx                ; lpString1
call    ds:lstrcatA
push    offset aLab1201_dll ; "Lab12-01.dll"
lea     eax, [ebp+Buffer]
push    eax                ; lpString1
call    ds:lstrcatA
lea     ecx, [ebp+var_1120]
push    ecx

```

发现它被设置成当前目录，并追加了 Lab12-01.dll 的一个路径。现在可以断定这个恶意代码将 Lab12-01.dll 的路径写入到 explorer.exe 进程中。

继续查看创建远程线程的代码部分：

```

.text:004012E0      push    offset ModuleName ; "kernel32.dll"
.text:004012E5      call    ds:GetModuleHandleA
.text:004012EB      mov     [ebp+nModule], eax
.text:004012F1      push    offset aLoadlibrarya ; "LoadLibraryA"
.text:004012F6      mov     eax, [ebp+hModule]
.text:004012FC      push    eax                ; hModule
.text:004012FD      call    ds:GetProcAddress
.text:00401303      mov     [ebp+lpStartAddress], eax
.text:00401309      push    0                ; lpThreadId
.text:0040130B      push    0                ; dwCreationFlags
.text:0040130D      mov     ecx, [ebp+lpBaseAddress]
.text:00401313      push    ecx                ; lpParameter
.text:00401314      mov     edx, [ebp+lpStartAddress]
.text:0040131A      push    eax                ; lpStartAddress
.text:0040131B      push    0                ; dwStackSize
.text:0040131D      push    0                ; lpThreadAttributes
.text:0040131F      mov     eax, [ebp+hProcess]
.text:00401325      push    eax                ; hProcess
.text:00401326      call    ds:CreateRemoteThread
.text:0040132C      mov     [ebp+var_1130], eax
.text:00401332      cmp     [ebp+var_1130], 0
.text:00401339      jnz     short loc_401340
.text:0040133B      or      eax, 0FFFFFFFh
.text:0040133E      jmp     short loc_401342

```

对 GetModuleHandleA 以及 GetProcAddress 的调用，被用来获取 LoadLibraryA 的地址。恶意代码中的 LoadLibraryA 的地址与 explorer.exe 中的地址是相同的。

可以看到 LoadLibraryA 的地址被写入到 lpStartAddress 中。lpstartAddress 提供给 CreateRemoteThread，将其作为参数调用 CreateRemoteThread 函数。反过来，在远程进程中启动一个线程，这个线程以参数 Lab12-01.dll 来调用 LoadLibraryA。

现在可以断定恶意代码执行了DLL注入，将 Lab12-01.dll 注入到 explorer.exe 中。

接下来分析dll文件：

可以看到其首先调用 CreateThread 函数创建线程

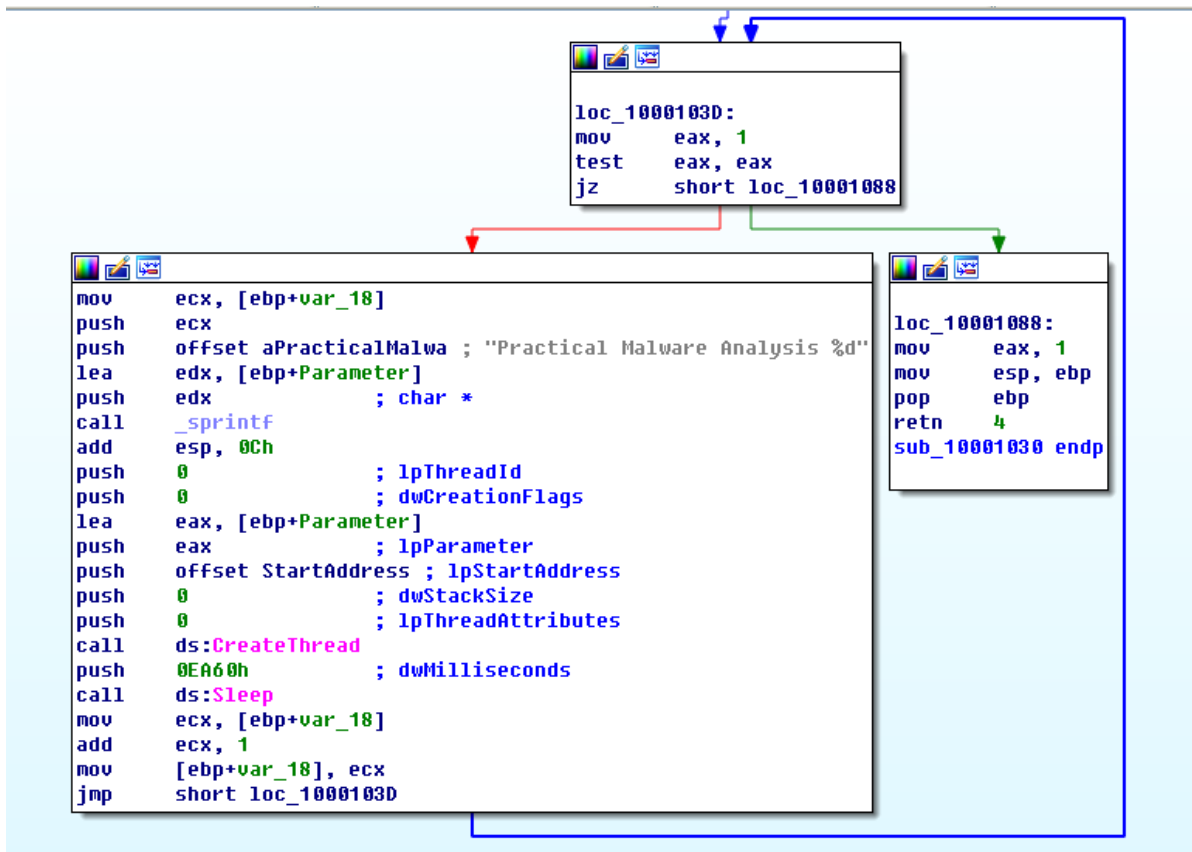
```
; BOOL __stdcall DllMain(HINSTANCE hinstDLL, DWORD fdwReason, LPVOID lpvReserved)
_DllMain@12 proc near

var_8= dword ptr -8
ThreadId= dword ptr -4
hinstDLL= dword ptr 8
fdwReason= dword ptr 0Ch
lpvReserved= dword ptr 10h

push    ebp
mov     ebp, esp
sub     esp, 8
cmp     [ebp+fdwReason], 1
jnz     short loc_100010C6
```

```
lea     eax, [ebp+ThreadId]
push    eax                ; lpThreadId
push    0                  ; dwCreationFlags
push    0                  ; lpParameter
push    offset sub_10001030 ; lpStartAddress
push    0                  ; dwStackSize
push    0                  ; lpThreadAttributes
call    ds:CreateThread
mov     [ebp+var_8], eax
```

下面分析其调用的 sub_10001030 函数。切换视图可以看到其为一个循环结构。循环体中同样调用 CreateThread 函数创建线程。接着以 60000 作为参数调用 Sleep 函数，即每次运行时间间隔一分钟。



查看 `StartAddress` 函数，可以看到其调用 `MessageBoxA` 函数创建消息提示框显示 Press OK to reboot，这与前面分析相吻合。

```

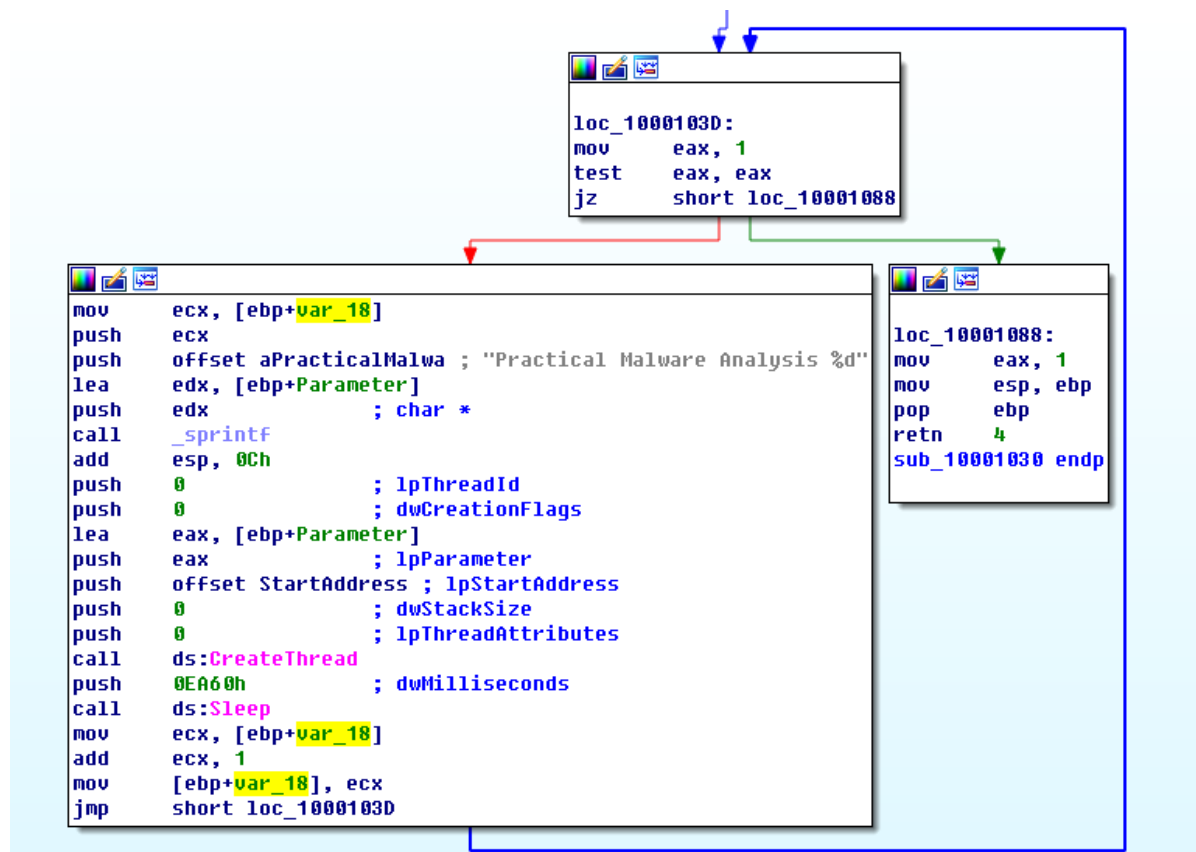
StartAddress proc near

lpCaption= dword ptr -4
lpThreadParameter= dword ptr 8

push    ebp
mov     ebp, esp
push    ecx
mov     eax, [ebp+lpThreadParameter]
mov     [ebp+lpCaption], eax
push    40040h ; uType
mov     ecx, [ebp+lpCaption]
push    ecx ; lpCaption
push    offset Text ; "Press OK to reboot"
push    0 ; hWnd
call    ds:MessageBoxA
mov     eax, 3
mov     esp, ebp
pop     ebp
retn    4
StartAddress endp

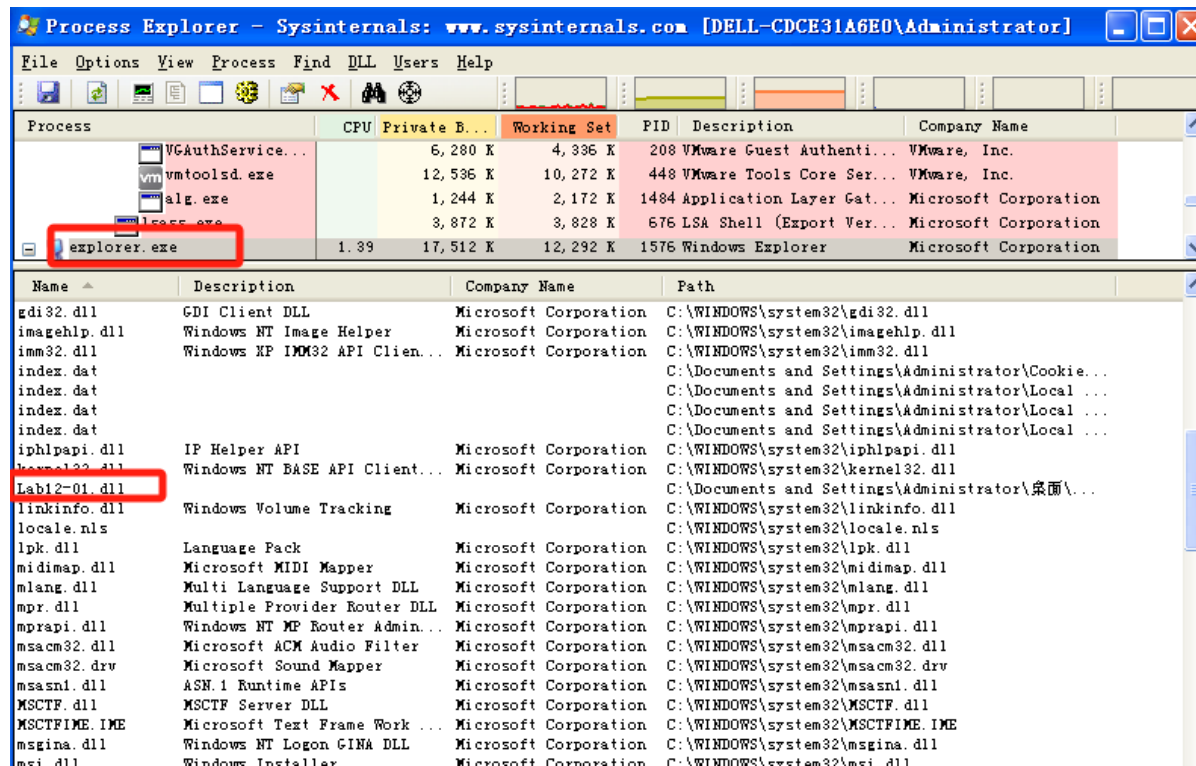
```

返回分析，可以看到 `var_18` 被赋初值为 0，运行后被被拼接到字符串后，每次循环自增。



现在尝试让恶意代码停止弹出窗口：

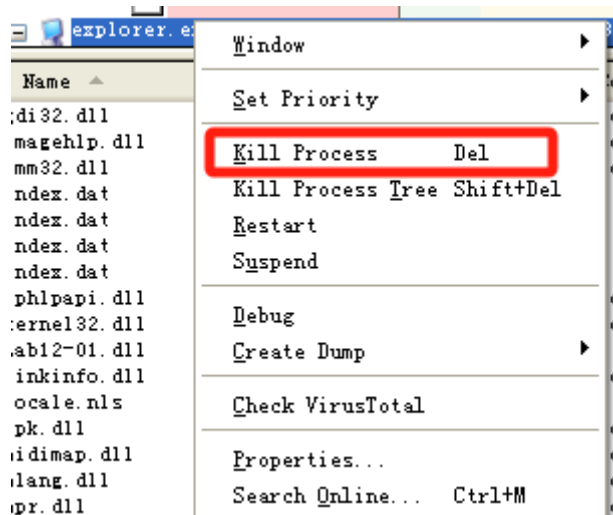
使用Process Explorer查看运行 explorer.exe 程序加载的dll：



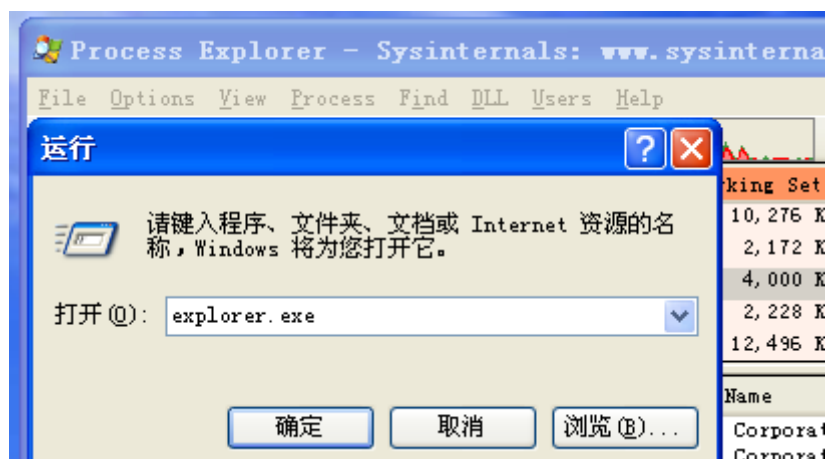
可以看到explorer.exe内存空间中加载了 Lab12-01.dll。

Process Explorer是一个发现DLL注入的简单办法，并且确认了先前的IDA Pro分析。

停止这些弹出框，可以使用ProcessExplorer来杀死explorer.exe



然后通过选择File→Run，输入explorer重启它



可以发现不再弹出消息框，并且explorer.exe程序下也不再加载 Lab12-01.dll。

explorer.exe	1.41	8,220 K	12,772 K	1368 Windows Explorer	Microsoft Corporation
rundll32.exe		3,016 K	5,296 K	3408 Run a DLL as an App	Microsoft Corporation

Name	Description	Company Name	Path
crypt32.dll	Crypto API32	Microsoft Corporation	C:\WINDOWS\system32\crypt32.dll
cryptui.dll	Microsoft Trust UI Provider	Microsoft Corporation	C:\WINDOWS\system32\cryptui.dll
cscdll.dll	Offline Network Agent	Microsoft Corporation	C:\WINDOWS\system32\cscdll.dll
cscui.dll	Client Side Caching UI	Microsoft Corporation	C:\WINDOWS\system32\cscui.dll
ctype.nls			C:\WINDOWS\system32\ctype.nls
dot3api.dll	802.3 自动配置 API	Microsoft Corporation	C:\WINDOWS\system32\dot3api.dll
dot3dlg.dll	802.3 UI 帮助程序	Microsoft Corporation	C:\WINDOWS\system32\dot3dlg.dll
eapcfg.dll	EAP 对等配置	Microsoft Corporation	C:\WINDOWS\system32\eapcfg.dll
eappprxy.dll	Microsoft EAPHost Peer Cli...	Microsoft Corporation	C:\WINDOWS\system32\eappprxy.dll
explorer.exe	Windows Explorer	Microsoft Corporation	C:\WINDOWS\explorer.exe
gdi32.dll	GDI Client DLL	Microsoft Corporation	C:\WINDOWS\system32\gdi32.dll
imagehlp.dll	Windows NT Image Helper	Microsoft Corporation	C:\WINDOWS\system32\imagehlp.dll
imm32.dll	Windows XP IMM32 API Clie...	Microsoft Corporation	C:\WINDOWS\system32\imm32.dll
iphlpapi.dll	IP Helper API	Microsoft Corporation	C:\WINDOWS\system32\iphlpapi.dll
kernel32.dll	Windows NT BASE API Client...	Microsoft Corporation	C:\WINDOWS\system32\kernel32.dll
linkinfo.dll	Windows Volume Tracking	Microsoft Corporation	C:\WINDOWS\system32\linkinfo.dll
locale.nls			C:\WINDOWS\system32\locale.nls
lpk.dll	Language Pack	Microsoft Corporation	C:\WINDOWS\system32\lpk.dll
midimap.dll	Microsoft MIDI Mapper	Microsoft Corporation	C:\WINDOWS\system32\midimap.dll
msacm32.dll	Microsoft ACM Audio Filter	Microsoft Corporation	C:\WINDOWS\system32\msacm32.dll
msacm32.drv	Microsoft Sound Mapper	Microsoft Corporation	C:\WINDOWS\system32\msacm32.drv
msasn1.dll	ASN.1 Runtime APIs	Microsoft Corporation	C:\WINDOWS\system32\msasn1.dll
MSCTF.dll	MSCTF Server DLL	Microsoft Corporation	C:\WINDOWS\system32\MSCTF.dll
MSCTFIME.IME	Microsoft Text Frame Work ...	Microsoft Corporation	C:\WINDOWS\system32\MSCTFIME.IME
msi.dll	Windows Installer	Microsoft Corporation	C:\WINDOWS\system32\msi.dll
msimg32.dll	GDIEXT Client DLL	Microsoft Corporation	C:\WINDOWS\system32\msimg32.dll
msvcp60.dll	Microsoft (R) C++ Runtime ...	Microsoft Corporation	C:\WINDOWS\system32\msvcp60.dll
msvcr7.dll	Windows NT CRT DLL	Microsoft Corporation	C:\WINDOWS\system32\msvcr7.dll
netapi32.dll	Net Win32 API DLL	Microsoft Corporation	C:\WINDOWS\system32\netapi32.dll
netshell.dll	Network Connections Shell	Microsoft Corporation	C:\WINDOWS\system32\netshell.dll

Address	Ordinal	Name	Library
00000000...		CloseHandle	KERNEL32
00000000...		VirtualFree	KERNEL32
00000000...		ReadFile	KERNEL32
00000000...		VirtualAlloc	KERNEL32
00000000...		GetFileSize	KERNEL32
00000000...		CreateFileA	KERNEL32
00000000...		ResumeThread	KERNEL32
00000000...		SetThreadContext	KERNEL32
00000000...		WriteProcessMemory	KERNEL32
00000000...		VirtualAllocEx	KERNEL32
00000000...		GetProcAddress	KERNEL32
00000000...		GetModuleHandleA	KERNEL32
00000000...		ReadProcessMemory	KERNEL32
00000000...		GetThreadContext	KERNEL32
00000000...		CreateProcessA	KERNEL32
00000000...		FreeResource	KERNEL32
00000000...		SizeofResource	KERNEL32
00000000...		LockResource	KERNEL32
00000000...		LoadResource	KERNEL32
00000000...		FindResourceA	KERNEL32
00000000...		GetSystemDirectoryA	KERNEL32
00000000...		Sleep	KERNEL32
00000000...		GetCommandLineA	KERNEL32
00000000...		GetVersion	KERNEL32
00000000...		ExitProcess	KERNEL32
00000000...		TerminateProcess	KERNEL32
00000000...		GetCurrentProcess	KERNEL32
00000000...		UnhandledExceptionFilter	KERNEL32
00000000...		GetModuleFileNameA	KERNEL32
00000000...		FreeEnvironmentStringsA	KERNEL32
00000000...		FreeEnvironmentStringsW	KERNEL32
00000000...		WideCharToMultiByte	KERNEL32
00000000...		GetEnvironmentStrings	KERNEL32

CreateProcessA、GetThreadContext 以及 SetThreadContext 暗示这个程序创建新的进程，并修改进程中的线程执行上下文。导入函数 ReadProcessMemory 和 WriteProcessMemory 可知这个程序对进程内存空间进行了直接读写。导入函数 LockResource 和 SizeOfResource 可知这个进程比较重要的数据可能保存在哪。

分析调用 CreateProcessA 函数的目的：

```

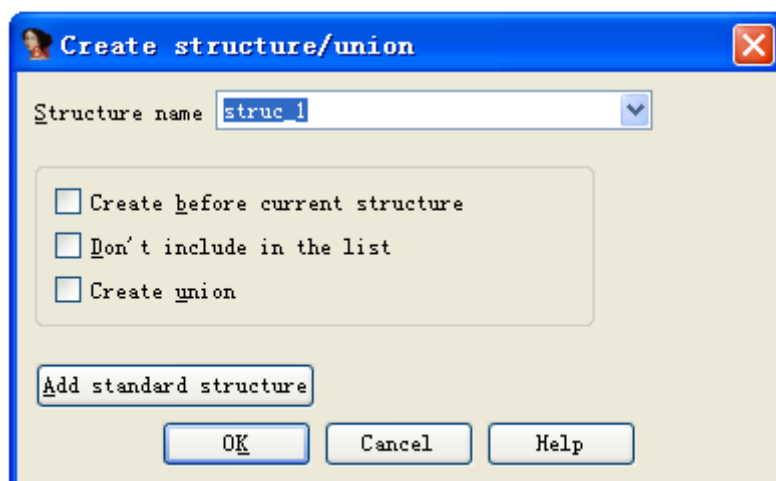
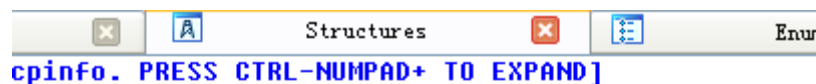
.text:00401145      lea     edx, [ebp+ProcessInformation]
.text:00401148      push    edx                ; lpProcessInformation
.text:00401149      lea     eax, [ebp+StartupInfo]
.text:0040114C      push    eax                ; lpStartupInfo
.text:0040114D      push    0                  ; lpCurrentDirectory
.text:0040114F      push    0                  ; lpEnvironment
.text:00401151      push    4                  ; dwCreationFlags
.text:00401153      push    0                  ; bInheritHandles
.text:00401155      push    0                  ; lpThreadAttributes
.text:00401157      push    0                  ; lpProcessAttributes
.text:00401159      push    0                  ; lpCommandLine
.text:0040115B      mov     ecx, [ebp+lpApplicationName]
.text:0040115E      push    ecx                ; lpApplicationName
.text:0040115F      call    ds:CreateProcessA
.text:00401165      test    eax, eax
.text:00401167      jz      loc_401313
.text:0040116D      push    4                  ; flProtect
.text:0040116F      push    1000h              ; flAllocationType
.text:00401174      push    200h               ; dwSize
.text:00401179      push    0                  ; lpAddress
.text:0040117B      call    ds:VirtualAlloc
.text:00401181      mov     [ebp+lpContext], eax
.text:00401184      mov     edx, [ebp+lpContext]
.text:00401187      mov     dword ptr [edx], 10007h
.text:0040118D      mov     eax, [ebp+lpContext]
.text:00401190      push    eax                ; lpContext
.text:00401191      mov     ecx, [ebp+ProcessInformation.hThread]
.text:00401194      push    ecx                ; hThread
.text:00401195      call    ds:GetThreadContext

```

可以看到一个 `push 4` 指令，而它被 IDAPro 标记为参数 `dwCreationFlags`。通过 MSDN 文档对 `CreateProcess` 的描述可知这是 `CREATE_SUSPENDED` 标志，它允许进程被创建但并不启动。这个进程将不会执行，除非等这个主进程调用 `APIResumeThread` 函数时，它才会被启动。

并且可以看到这个程序正在访问一个线程上下文。`GetThreadContext` 的 `hThread` 参数与传递给 `CreateProcessA` 的参数处于同一缓冲区，可知这个程序正在访问挂起进程的上下文。获取进程句柄非常重要，因为程序将使用这个进程句柄与挂起进程进行交互。

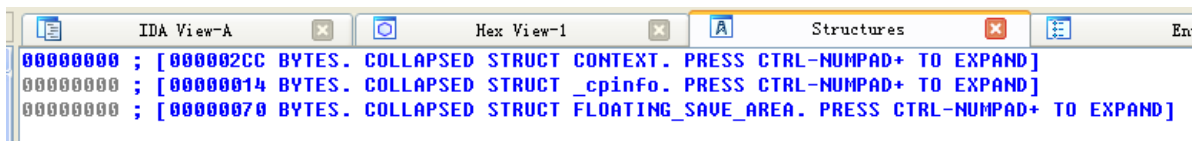
为了更好地判断这个程序用进程上下文做了什么，需要在 IDAPro 中添加 `CONTEXT` 结构体。



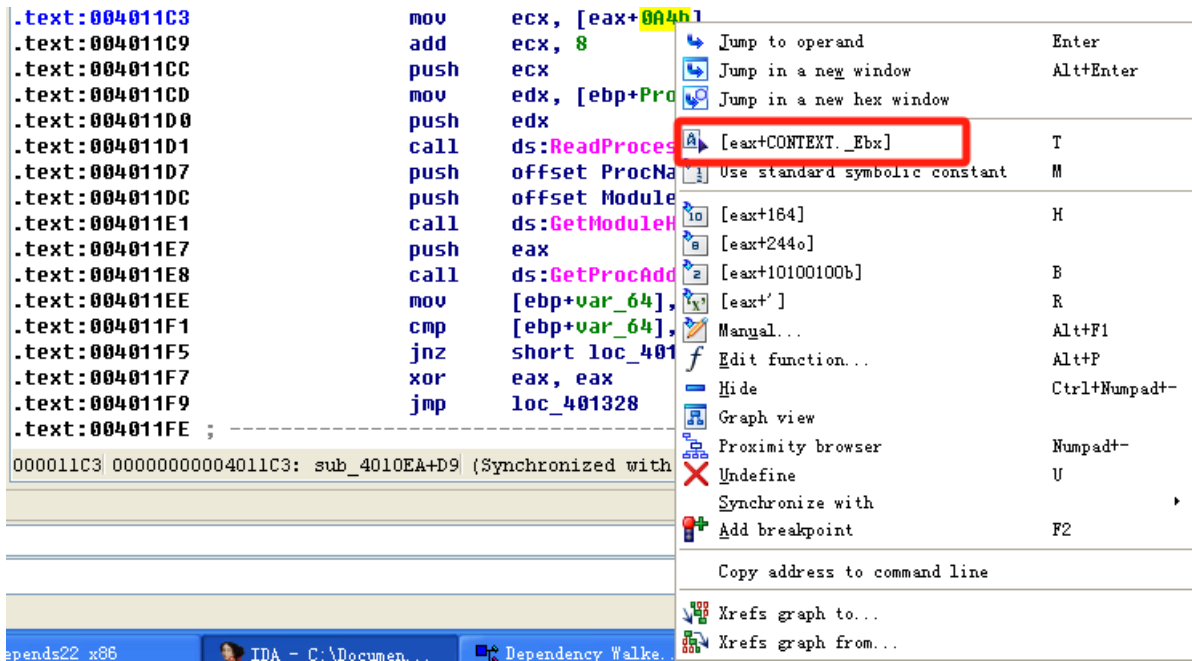
并放置名为 `CONTEXT` 的结构体：

CONTENTRESTRICTION	struct tagCONTENTRESTRICTION	MS SDK (Windows XP)
CONTEXT	struct _CONTEXT	MS SDK (Windows XP)
CONTEXTMENUITEM	struct _CONTEXTMENUITEM	MS SDK (Windows XP)
CONTEXTMENUITEM2	struct _CONTEXTMENUITEM2	MS SDK (Windows XP)
CONTEXTMENUITEM3	struct _CONTEXTMENUITEM3	MS SDK (Windows XP)

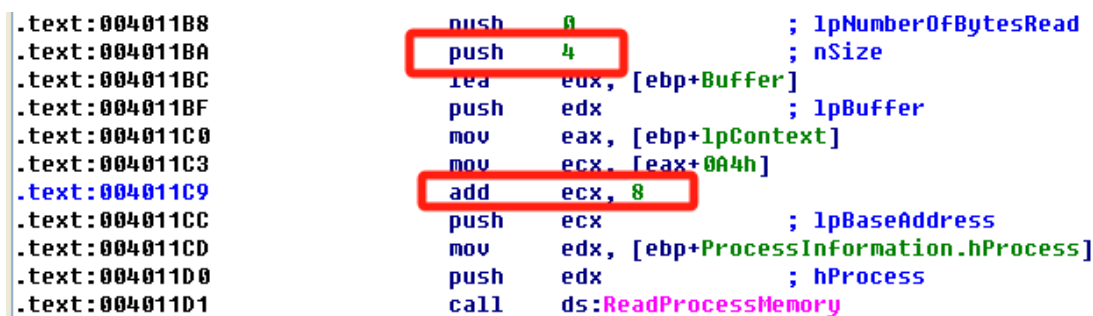
添加成功



右击位置0x004011C3来解析这个结构体的偏移:



可以看到, 偏移0xA4实际上通过 [eax+CONTEXT._Ebx] 来引用这个进程的EBX寄存器。



这个新创建就被挂起的进程EBX寄存器总是包含一个指向进程环境块(PEB)的数据结构。可以看到程序以8字节递增结构体, 并将这个值压到栈上, 作为要读取内存的起始地址。

使用互联网搜索PEB数据结构的8字节偏移处: 一个指向mageBaseAddress(被加载的可执行文件起始部分)的指针。将这个地址作为读取位置, 并在处读取4个字节, 可以看到IDA Pro已标记为lpBuffer的变量将包含被挂起进程的ImageBase。

这个程序使用在0x004011E8处的GetProcAddress, 手动解析导入函数UnMapViewOfSection, 并且在0x004011FE处, ImageBaseAddress作为UnMapViewOfSection的一个参数传入。

UnMapViewOfSection的调用从内存中移除这个被挂起进程, 此时这个程序将不再执行。

参数被压到栈上, 然后调用VirtualAllocEx:


```

.text:00401209      push     40h                ; flProtect
.text:0040120B      push     3000h             ; flAllocationType
.text:00401210      mov     edx, [ebp+var_8]
.text:00401213      mov     eax, [edx+50h]
.text:00401216      push     eax                ; dwSize
.text:00401217      mov     ecx, [ebp+var_8]
.text:0040121A      mov     edx, [ecx+34h]
.text:0040121D      push     edx                ; lpAddress
.text:0040121E      mov     eax, [ebp+ProcessInformation.hProcess]
.text:00401221      push     eax                ; hProcess
.text:00401222      call    ds:VirtualAllocEx

```

显示了这个程序在被挂起进程的地址空间中分配内存。

```

mov     dx, [ecx]
cmp     edx, 'ZM'
jnz     loc_40131F

```



```

mov     eax, [ebp+var_4]
mov     ecx, [ebp+lpBuffer]
add     ecx, [eax+3Ch]
mov     [ebp+var_8], ecx
mov     edx, [ebp+var_8]
cmp     dword ptr [edx], 'EP'
jnz     loc_401319

```

在这个函数的开头，程序检查在0x004010FE处的MZ和在0x00401119处的PE。如果这个检查是有效的，可以知道var_8是一个指针，指向加载到内存的PE头。

之后将 var_8 存储在到 ecx 中，再将 ecx 加上34h

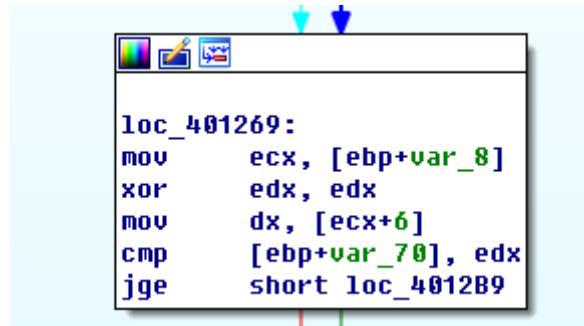
```

loc_4011FE:
mov     eax, [ebp+Buffer]
push    eax
mov     ecx, [ebp+ProcessInformation.hProcess]
push    ecx
call    [ebp+var_64]
push    40h                ; flProtect
push    3000h             ; flAllocationType
mov     edx, [ebp+var_8]
mov     eax, [edx+50h]
push    eax                ; dwSize
mov     ecx, [ebp+var_8]
mov     edx, [ecx+34h]
push    edx                ; lpAddress
mov     eax, [ebp+ProcessInformation.hProcess]
push    eax                ; hProcess
call    ds:VirtualAllocEx
mov     [ebp+lpBaseAddress], eax
cmp     [ebp+lpBaseAddress], 0
jz      loc_401307

```

通过查找PE文件结构图发现从基址加上 34h 就是映像基址的位置，即将映像基址存入 edx 中。而 edx 会作为 lpaddress 参数传入 VirtualAllocEx；另外还有 [edx+50h] 一同移入其中，经过分析可以发现其为内存中映像总大小 dwSize。调用函数，向内存中写入数据。推测该部分移动一个 PE 文件到另一个内存地址空间去。

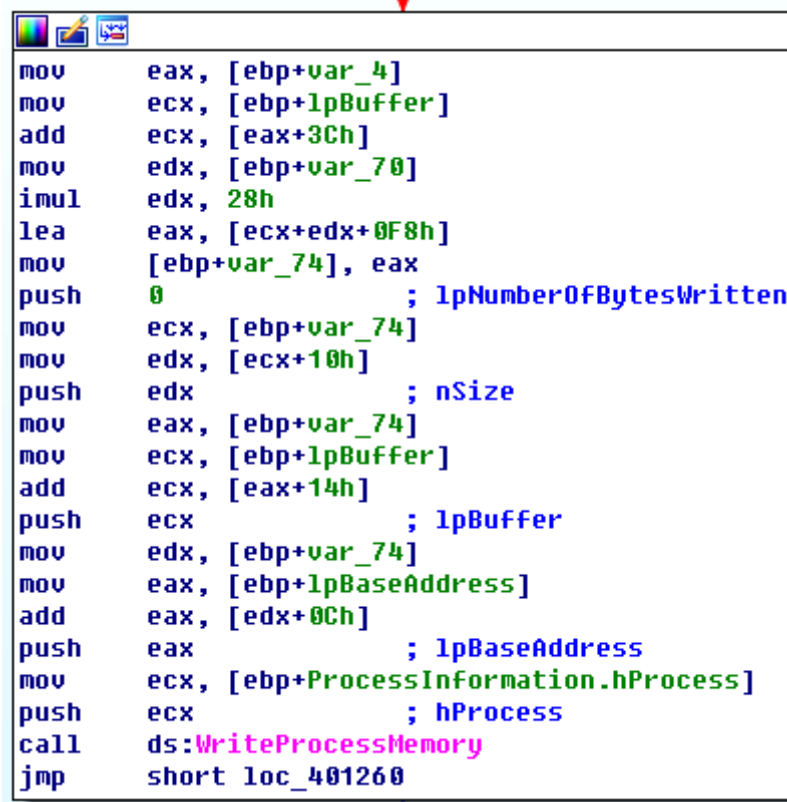
继续分析，可以看到其在 ecx 的基础上加上了 6 偏移后放入 edx 寄存器中



```
loc_401269:
mov     ecx, [ebp+var_8]
xor     edx, edx
mov     dx, [ecx+6]
cmp     [ebp+var_70], edx
jge     short loc_4012B9
```

现在此寄存器中存储的即为区段数。说明这段循环复制 PE 文件的可执行段到挂起进程里面。

继续分析，var_4 指向的是 PE 文件中 MZ 的位置，偏移 3C 后即指向 PE 标志位的偏移，故而该位置保存 PE 文件的偏移位标识，可以据此获获取 PE 文件头的位置，此时 ecx 中保存的即为 PE 文件头的位置。



```
mov     eax, [ebp+var_4]
mov     ecx, [ebp+lpBuffer]
add     ecx, [eax+3Ch]
mov     edx, [ebp+var_70]
imul    edx, 28h
lea     eax, [ecx+edx+0F8h]
mov     [ebp+var_74], eax
push    0 ; lpNumberOfBytesWritten
mov     ecx, [ebp+var_74]
mov     edx, [ecx+10h]
push    edx ; nSize
mov     eax, [ebp+var_74]
mov     ecx, [ebp+lpBuffer]
add     ecx, [eax+14h]
push    ecx ; lpBuffer
mov     edx, [ebp+var_74]
mov     eax, [ebp+lpBaseAddress]
add     eax, [edx+0Ch]
push    eax ; lpBaseAddress
mov     ecx, [ebp+ProcessInformation.hProcess]
push    ecx ; hProcess
call    ds:WriteProcessMemory
jmp     short loc_401260
```

继续分析，可以看到其调用了 SetThreadContext 函数，修改 eax 寄存器中的数据，将 eax 中的值设置为可执行文件的加载入口点。

```

loc_4012B9:                ; lpNumberOfBytesWritten
push    0
push    4                  ; nSize
mov     edx, [ebp+var_8]
add     edx, 34h
push    edx                ; lpBuffer
mov     eax, [ebp+lpContext]
mov     ecx, [eax+0A4h]
add     ecx, 8
push    ecx                ; lpBaseAddress
mov     edx, [ebp+ProcessInformation.hProcess]
push    edx                ; hProcess
call    ds:WriteProcessMemory
mov     eax, [ebp+var_8]
mov     ecx, [ebp+lpBaseAddress]
add     ecx, [eax+28h]
mov     edx, [ebp+lpContext]
mov     [edx+0B0h], ecx
mov     eax, [ebp+lpContext]
push    eax                ; lpContext
mov     ecx, [ebp+ProcessInformation.hThread]
push    ecx                ; hThread
call    ds:SetThreadContext
mov     edx, [ebp+ProcessInformation.hThread]
push    edx                ; hThread
call    ds:ResumeThread
jmp     short loc_40130B

```

接着其调用 `ResumeThread` 函数，将进程进行替换。向前查找，可以看到是将 `svchost.exe` 文件作为参数传递给了 `sub_40149D` 函数。

```

.text:004014FC             call    ds:GetModuleHandleA
.text:00401502             mov     [ebp+hModule], eax
.text:00401508             push    400h                ; uSize
.text:0040150D             lea     eax, [ebp+ApplicationName]
.text:00401513             push    eax                ; lpBuffer
.text:00401514             push    offset aSvchost_exe ; "\\svchost.exe"
.text:00401519             call    sub_40149D
.text:0040151E             add     esp, 0Ch
.text:00401521             mov     ecx, [ebp+hModule]

```

步入分析 `sub_40149D` 函数，发现其调用了获取系统路径函数并且调用了 `strcat` 字符串连接函数。综合，推测其目的为构造 `svchost.exe` 路径，即有进程替换了 `svchost.exe`。

```

.text:0040149D sub_40149D proc near ; CODE XREF: _main+39Jp
.text:0040149D
.text:0040149D arg_0 = dword ptr 8
.text:0040149D lpBuffer = dword ptr 0Ch
.text:0040149D uSize = dword ptr 10h
.text:0040149D
.text:0040149D push ebp
.text:0040149E mov ebp, esp
.text:004014A0 mov eax, [ebp+uSize]
.text:004014A3 push eax ; uSize
.text:004014A4 mov ecx, [ebp+lpBuffer]
.text:004014A7 push ecx ; lpBuffer
.text:004014A8 call ds:GetSystemDirectoryA
.text:004014AE mov edx, [ebp+lpBuffer]
.text:004014B1 push edx ; char *
.text:004014B2 call _strlen
.text:004014B7 add esp, 4
.text:004014BA mov ecx, [ebp+uSize]
.text:004014BD sub ecx, eax
.text:004014BF push ecx ; size_t
.text:004014C0 mov edx, [ebp+arg_0]
.text:004014C3 push edx ; char *
.text:004014C4 mov eax, [ebp+lpBuffer]
.text:004014C7 push eax ; char *
.text:004014C8 call _strlen
.text:004014CD add esp, 4
.text:004014D0 mov ecx, [ebp+lpBuffer]
.text:004014D3 add ecx, eax
.text:004014D5 push ecx ; char *
.text:004014D6 call _strncat
.text:004014DB add esp, 0Ch
.text:004014DE pop ebp
.text:004014DF ret

```

函数 sub_40132C 调用函数 FindResource、LoadResource、LockResource、SizeOfResource、VirtualAlloc 以及 memcpy。

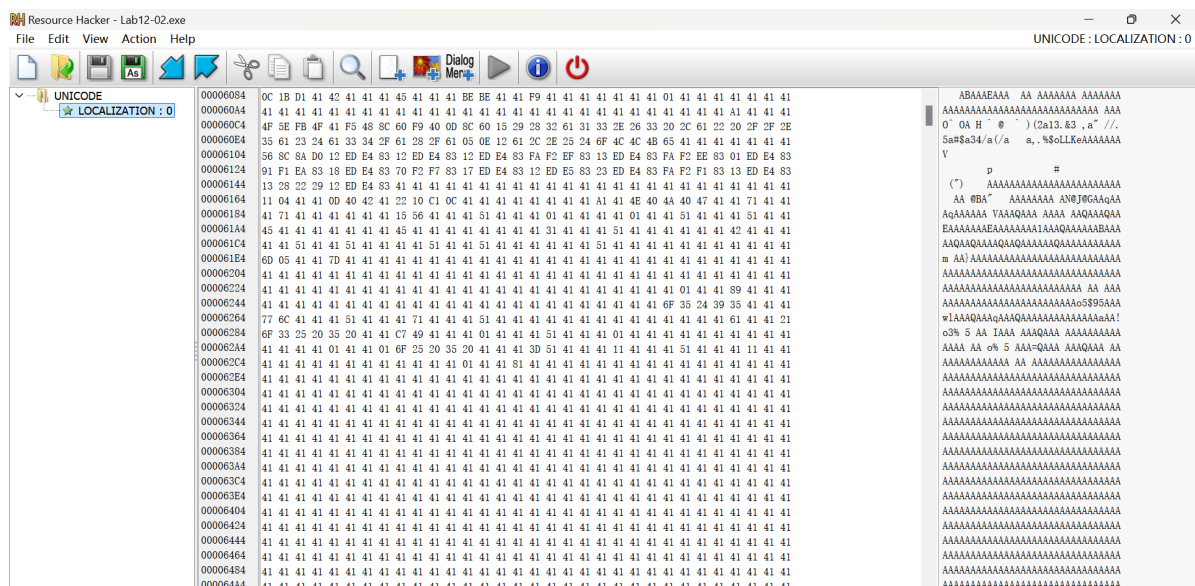
```

.text:00401521 mov ecx, [ebp+hModule]
.text:00401527 push ecx ; hModule
.text:00401528 call sub_40132C
.text:0040152D add esp, 4
.text:00401530 mov [ebp+lpAddress], eax

```

这个程序从可执行文件的资源段复制数据到内存中。

使用Resource Hacker来查看在资源段中的项



在0x00401425处，看到这个缓冲区被传递给函数sub_401000，这个函数看起来像一个XOR例程

```

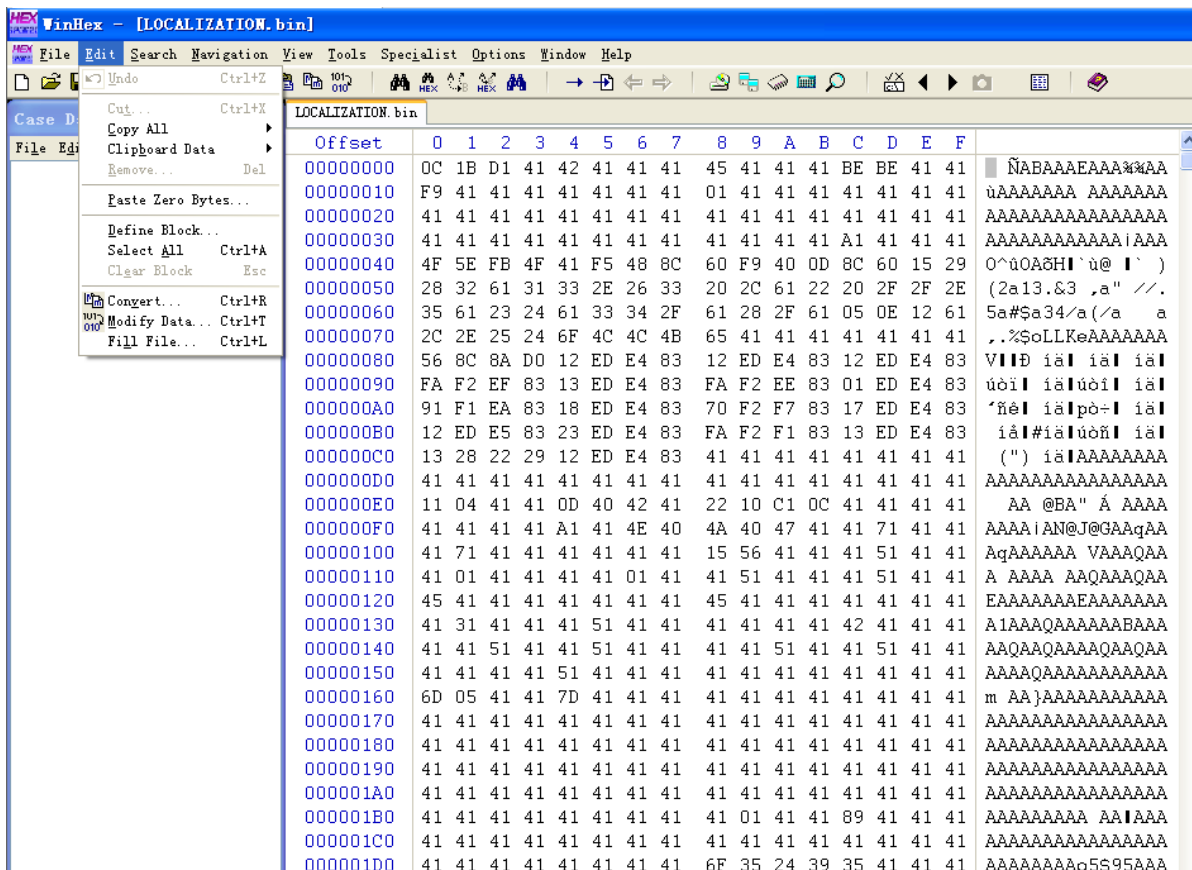
.text:00401000      push    ebp
.text:00401001      mov     ebp, esp
.text:00401003      push    ecx
.text:00401004      mov     [ebp+var_4], 0
.text:00401008      jmp     short loc_401016
.text:0040100D      ; -----
.text:0040100D      loc_40100D:                                ; CODE XREF: sub_401000+31↓j
.text:0040100D      mov     eax, [ebp+var_4]
.text:00401010      add     eax, 1
.text:00401013      mov     [ebp+var_4], eax
.text:00401016      loc_401016:                                ; CODE XREF: sub_401000+B↑j
.text:00401016      mov     ecx, [ebp+var_4]
.text:00401019      cmp     ecx, [ebp+arg_4]
.text:0040101C      jnb     short loc_401033
.text:0040101E      mov     edx, [ebp+arg_0]
.text:00401021      add     edx, [ebp+var_4]
.text:00401024      mov     al, [edx]
.text:00401026      xor     al, [ebp+arg_8]
.text:00401029      mov     ecx, [ebp+arg_0]
.text:0040102C      add     ecx, [ebp+var_4]
.text:0040102F      mov     [ecx], al
.text:00401031      jmp     short loc_40100D

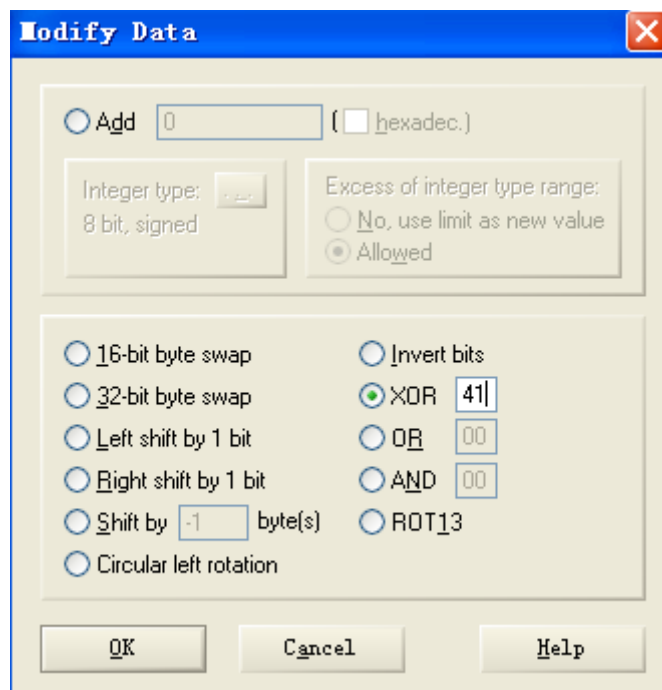
```

```

.text:0040141B loc_40141B:                                     ; CODE XREF: sub_40132C+E0↑j
.text:0040141B          push     41h
.text:0040141D          mov     edx, [ebp+dwSize]
.text:00401420          push     edx
.text:00401421          mov     eax, [ebp+var_8]
.text:00401424          push     eax
.text:00401425          call    sub_401000
.text:0040142A          add     esp, 0Ch

```





在执行这个转换以后，得到了一个有效的PE可执行文件：

LOCALIZATION. bin																	
Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
00000000	4D	5A	90	00	03	00	00	00	04	00	00	00	FF	FF	00	00	MZ I yy
00000010	B8	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	, @
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000030	00	00	00	00	00	00	00	00	00	00	00	00	E0	00	00	00	à
00000040	0E	1F	BA	0E	00	B4	09	CD	21	B8	01	4C	CD	21	54	68	é ' í!, Lí!Th
00000050	69	73	20	70	72	6F	67	72	61	6D	20	63	61	6E	6E	6F	is program canno
00000060	74	20	62	65	20	72	75	6E	20	69	6E	20	44	4F	53	20	t be run in DOS
00000070	6D	6F	64	65	2E	0D	0D	0A	24	00	00	00	00	00	00	00	mode. \$
00000080	17	CD	CB	91	53	AC	A5	C2	53	AC	A5	C2	53	AC	A5	C2	ÍË'S-ŴAS-ŴAS-ŴA
00000090	BB	B3	AE	C2	52	AC	A5	C2	BB	B3	AF	C2	40	AC	A5	C2	»³@ÄR-ŴA»³-Ä@-ŴA
000000A0	D0	B0	AB	C2	59	AC	A5	C2	31	B3	B6	C2	56	AC	A5	C2	Đ°«ÄY-ŴA1³ŴAV-ŴA
000000B0	53	AC	A4	C2	62	AC	A5	C2	BB	B3	B0	C2	52	AC	A5	C2	S-ŴAb-ŴA»³°ÄR-ŴA
000000C0	52	69	63	68	53	AC	A5	C2	00	00	00	00	00	00	00	00	RichS-ŴA
000000D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000000E0	50	45	00	00	4C	01	03	00	63	51	80	4D	00	00	00	00	PE L cQIM
000000F0	00	00	00	00	E0	00	0F	01	0B	01	06	00	00	30	00	00	à 0
00000100	00	30	00	00	00	00	00	00	54	17	00	00	00	10	00	00	0 T
00000110	00	40	00	00	00	00	40	00	00	10	00	00	00	10	00	00	@ @
00000120	04	00	00	00	00	00	00	00	04	00	00	00	00	00	00	00	
00000130	00	70	00	00	00	10	00	00	00	00	00	00	03	00	00	00	p
00000140	00	00	10	00	00	10	00	00	00	00	10	00	00	10	00	00	
00000150	00	00	00	00	10	00	00	00	00	00	00	00	00	00	00	00	
00000160	2C	44	00	00	3C	00	00	00	00	00	00	00	00	00	00	00	,D <
00000170	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000180	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000190	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000001A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

这个执行文件后续被用来替换svchost.exe进程实例。

2. 问题解答

(1) 这个程序的目的是什么？

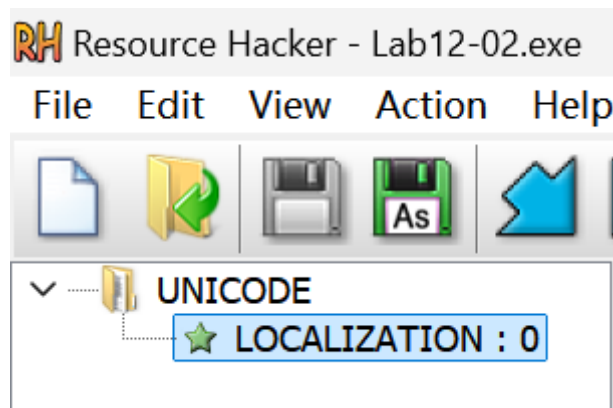
该恶意代码隐蔽启动另一个程序。

(2) 启动器恶意代码是如何隐蔽执行的？

其进行进程替换进行秘密执行，即先将正常程序挂起，然后逐模块替换。

(3) 恶意代码的负载存储在哪里？

这个恶意的有效载荷(payload)被保存在这个程序的资源节中。这个资源节的类型是 UNICODE,且名字是 LOCALIZATION。



(4) 恶意负载是如何被保护的？

保存在这个程序资源节中的恶意有效载荷是经过 XOR编码过的。这个解码例程可以在 sub_40132C 处找到，而XOR字节在 0x0040141B处可以找到。

(5) 字符串列表是如何被保护的？

这些字符串是使用在sub_401000处的函数，来进行XOR编码的。

Lab12-03

1.过程分析

使用IDA Pro查看其导入函数：

000000...	GetForegroundWindow	USER32
000000...	GetWindowTextA	USER32
000000...	CallNextHookEx	USER32
000000...	FindWindowA	USER32
000000...	ShowWindow	USER32
000000...	SetWindowsHookExA	USER32
000000...	IsMessageInQueue	USER32
000000...	UnhookWindowsHookEx	USER32

SetWindowsHookExA 是一个允许应用程序挂钩或监控微软Windows内部事件的API

看到 SetWindowsHookExA 从main中被调用：

```

.text:00401035 loc_401035:                                ; CODE XREF: _main+27↑j
.text:00401035      push     400h                                ; size_t
.text:0040103A      push     1                                    ; int
.text:0040103C      push     offset byte_405350 ; void *
.text:00401041      call    _memset
.text:00401046      add     esp, 0Ch
.text:00401049      push     0                                    ; dwThreadId
.text:0040104B      push     0                                    ; lpModuleName
.text:0040104D      call    ds:GetModuleHandleA
.text:00401053      push     eax                                    ; hmod
.text:00401054      push     offset fn                                ; lpfn
.text:00401059      push     0Dh                                    ; idHook
.text:0040105B      call    ds:SetWindowsHookExA
.text:00401061      mov     [ebp+hhk], eax

```

MSDN文档显示第一个参数0Dh，对应 WH_KEYBOARD_LL，它使用IDAPro标记为 lpfn 的挂钩函数启用键盘事件监控。这个程序可能对键击消息做某些手脚，而这个fn函数正在接收这些击键记录。

那么继续查看fn函数：

```

.text:00401086      push     ebp |
.text:00401087      mov     ebp, esp
.text:00401089      cmp     [ebp+code], 0
.text:0040108D      jnz     short loc_4010AF
.text:0040108F      cmp     [ebp+wParam], 104h
.text:00401096      jz      short loc_4010A1
.text:00401098      cmp     [ebp+wParam], 100h
.text:0040109F      jnz     short loc_4010AF
.text:004010A1      loc_4010A1:                                ; CODE XREF: fn+10↑j
.text:004010A1      mov     eax, [ebp+lParam]
.text:004010A4      mov     ecx, [eax]
.text:004010A6      push     eax                                    ; Buffer
.text:004010A7      call    sub_4010C7
.text:004010AC      add     esp, 4
.text:004010AF      loc_4010AF:                                ; CODE XREF: fn+7↑j
                                           ; fn+19↑j
.text:004010AF      mov     edx, [ebp+lParam]
.text:004010B2      push     edx                                    ; lParam
.text:004010B3      mov     eax, [ebp+wParam]
.text:004010B6      push     eax                                    ; wParam
.text:004010B7      mov     ecx, [ebp+code]
.text:004010BA      push     ecx                                    ; nCode
.text:004010BB      push     0                                    ; hhk
.text:004010BD      call    ds:CallNextHookEx
.text:004010C3      pop     ebp
.text:004010C4      retn    0Ch
.text:004010C4      fn

```

可以看到其带有三个参数。WH_KEYBOARD_LL 回调函数实际上是 LowLevelKeyboardProc 回调函数。接着在 0040108f, 00401098 处有两个 cmp 比较，涉及到的常量分别为 100h, 104h。

接着将虚拟按键码作为参数传到到函数 sub_4010C7 中，继续步入分析：

```

.text:004010C7      push     ebp
.text:004010C8      mov     ebp, esp
.text:004010CA      sub     esp, 0Ch
.text:004010CD      mov     [ebp+NumberOfBytesWritten], 0
.text:004010D4      push     0                                    ; hTemplateFile
.text:004010D6      push     80h                                ; dwFlagsAndAttributes
.text:004010DB      push     4                                    ; dwCreationDisposition
.text:004010DD      push     0                                    ; lpSecurityAttributes
.text:004010DF      push     2                                    ; dwShareMode
.text:004010E1      push     40000000h                          ; dwDesiredAccess
.text:004010E6      push     offset FileName ; "practicalmalwareanalysis.log"
.text:004010EB      call    ds:CreateFileA
.text:004010F1      mov     [ebp+hFile], eax
.text:004010F4      cmp     [ebp+hFile], 0FFFFFFFFh
.text:004010F8      jnz     short loc_4010FF
.text:004010FA      jmp     loc_40143D
.text:004010FF      : -----

```


可以看到其调用 CreateFileA 函数创建或打开了一个 log 文件。

```
.text:004010FF loc_4010FF:                                ; CODE XREF: sub_4010C7+31↑j
.text:004010FF      push     2                                ; dwMoveMethod
.text:00401101      push     0                                ; lpDistanceToMoveHigh
.text:00401103      push     0                                ; lDistanceToMove
.text:00401105      mov      eax, [ebp+hFile]
.text:00401108      push     eax                                ; hFile
.text:00401109      call     ds:SetFilePointer
.text:0040110F      push     400h                             ; nMaxCount
.text:00401114      push     offset Buffer - lpString
.text:00401119      call     ds:GetForegroundWindow
.text:0040111F      push     eax                                ; hWnd
.text:00401120      call     ds:GetWindowTextA
.text:00401126      push     offset Buffer, char *
.text:00401128      push     offset byte_405350 ; char *
.text:00401130      call     _strcmp
.text:00401135      add      esp, 8
.text:00401138      test     eax, eax
.text:0040113A      jz       short loc_4011AB
.text:0040113C      push     0                                ; lpOverlapped
.text:0040113E      lea      ecx, [ebp+NumberOfBytesWritten]
.text:00401141      push     ecx                                ; lpNumberOfBytesWritten
.text:00401142      push     0Ch                             ; nNumberOfBytesToWrite
.text:00401144      push     offset aWindow ; "\r\n[Window: "
.text:00401149      mov      edx, [ebp+hFile]
.text:0040114C      push     edx                                ; hFile
.text:0040114D      call     ds:WriteFile
.text:00401153      push     0                                ; lpOverlapped
.text:00401155      lea      eax, [ebp+NumberOfBytesWritten]
```

接着可以看到其调用 GetForegropundwindow 函数获取按键时的活动窗口，调用 GetWindowTextA 获取窗口标题，以此获取其上下文。

```
.text:00401202      mov      edx, [ebp+Buffer]
.text:00401205      mov      [ebp+var_C], edx
.text:00401208      mov      eax, [ebp+var_C]
.text:0040120B      sub      eax, 8
.text:0040120E      mov      [ebp+var_C], eax
.text:00401211      cmp      [ebp+var_C], 61h ; switch 98 cases
.text:00401215      ja       loc_40142C ; jumptable 00401226 default case
.text:00401218      mov      edx, [ebp+var_C]
.text:0040121E      xor      ecx, ecx
.text:00401220      mov      cl, ds:byte_40148D[edx]
.text:00401226      jmp      ds:off_401441[ecx*4] ; switch jump
```

这里的 var_C 由 Buffer 传入。

可以看到传入的参数实际上就是虚拟按键码。

继续分析，可以看到跳转表，在 00401220 看到虚拟按键码作为一个查询表的索引，查表得到的值作为跳转表 off_401441 的一个索引，加上当前的按键为 shift，其虚拟按键码为 0x10，回到 00401202 处重新分析。此时 var_C 为 0x10，而 0040120b 处将该值减去 8，它的值就变为了 8。

vac_C 中存的值为 8，在 byte_40148D 查找相应的偏移：


```

.text:00401480 byte_401480 db 0, 1, 12h, 12h ; DATA XREF: sub_4010C7+159↑r
.text:00401480 db 12h, 2, 12h, 12h ; indirect table for switch statement
.text:00401480 db 3, 4, 12h, 12h
.text:00401480 db 5, 12h, 12h, 12h
.text:00401480 db 12h, 12h, 12h, 12h
.text:00401480 db 12h, 12h, 12h, 12h
.text:00401480 db 6, 12h, 12h, 12h
.text:00401480 db 12h, 12h, 12h, 12h
.text:00401480 db 12h, 12h, 12h, 12h
.text:00401480 db 12h, 12h, 7, 12h
.text:00401480 db 12h, 12h, 12h, 12h
.text:00401480 db 12h, 12h, 12h, 12h
.text:00401480 db 12h, 12h, 12h, 12h
.text:00401480 db 12h, 12h, 12h, 12h
.text:00401480 db 12h, 12h, 12h, 12h
.text:00401480 db 12h, 12h, 12h, 12h
.text:00401480 db 12h, 12h, 12h, 12h
.text:00401480 db 12h, 12h, 12h, 12h
.text:00401480 db 8, 9, 0Ah, 0Bh
.text:00401480 db 0Ch, 0Dh, 0Eh, 0Fh
.text:00401480 db 10h, 11h
.text:004014EF align 10h

```

由于这是一个数组，偏移为 8，实际是第 9 个，也就是 3，然后根据地址 00401226 处的指令，将 $3*4 = 12$ 作为 off_401441 的偏移量。

```

loc_401249: ; CODE XREF: sub_4010C7+15F↑j
; DATA XREF: .text:off_401441↓o
push 0 ; jumptable 00401226 case 8
lea edx, [ebp+NumberOfBytesWritten]
push edx ; lpNumberOfBytesWritten
push 7 ; nNumberOfBytesToWrite
push offset aShift ; "[SHIFT]"
mov eax, [ebp+hFile]
push eax ; hFile
call ds:WriteFile
jmp loc_40142C ; jumptable 00401226 default case

```

发现其将 SHIFT 写入 log。

2. 问题解答

(1) 这个恶意负载的目的是什么？

该恶意代码的负载实现了一个击键记录器。

(2) 恶意负载是如何注入自身的？

该恶意代码使用 Hook 挂钩注入窃取击键记录。

(3) 这个程序还创建了哪些其他文件？

其创建了 practicalmalwareanalysis.log 日志文件，来保存击键记录。

Lab12-04

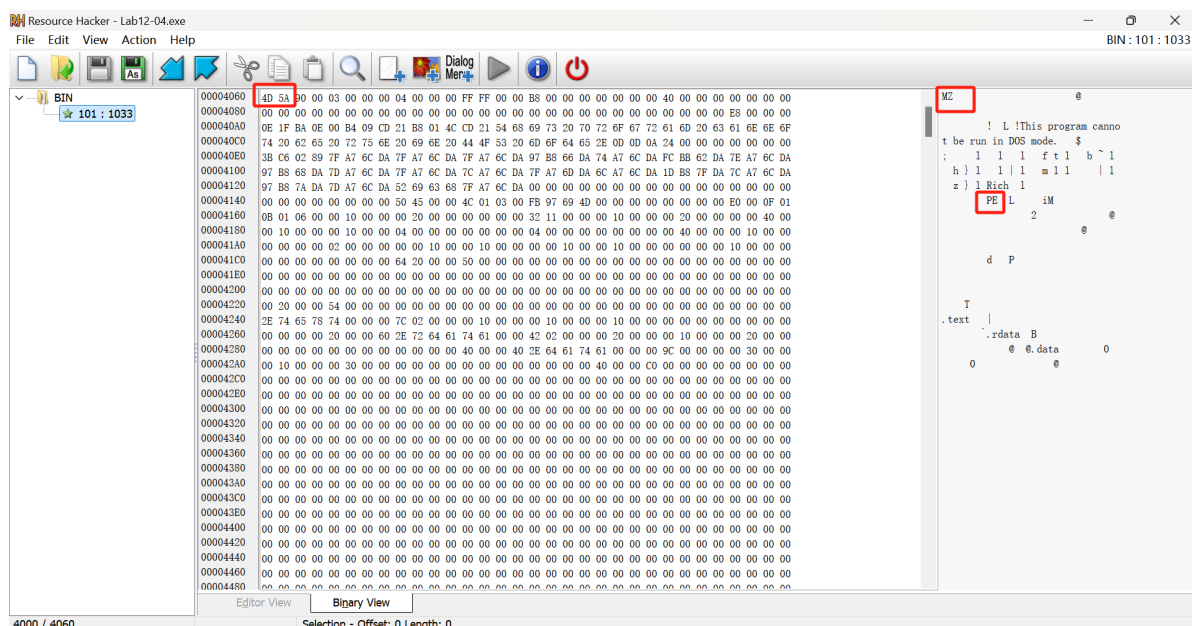
1. 过程分析

检查导入函数表：

Address	Ordinal	Name	Library
000000...		OpenProcessToken	ADVAPI32
000000...		LookupPrivilegeValueA	ADVAPI32
000000...		AdjustTokenPrivileges	ADVAPI32
000000...		GetProcAddress	KERNEL32
000000...		LoadLibraryA	KERNEL32
000000...		WinExec	KERNEL32
000000...		WriteFile	KERNEL32
000000...		CreateFileA	KERNEL32
000000...		SizeofResource	KERNEL32
000000...		CreateRemoteThread	KERNEL32
000000...		FindResourceA	KERNEL32
000000...		GetModuleHandleA	KERNEL32
000000...		GetWindowsDirectoryA	KERNEL32
000000...		MoveFileA	KERNEL32
000000...		GetTempPathA	KERNEL32
000000...		GetCurrentProcess	KERNEL32
000000...		OpenProcess	KERNEL32
000000...		CloseHandle	KERNEL32
000000...		LoadResource	KERNEL32
000000...		_snprintf	MSVCRT
000000...		_exit	MSVCRT
000000...		_XcptFilter	MSVCRT
000000...		exit	MSVCRT
000000...		_p__initenv	MSVCRT
000000...		__getmainargs	MSVCRT
000000...		_initterm	MSVCRT
000000...		__setusermatherr	MSVCRT
000000...		_adjust_fdiv	MSVCRT
000000...		_p__commode	MSVCRT
000000...		_p__fmode	MSVCRT
000000...		__set_app_type	MSVCRT
000000...		_except_handler3	MSVCRT
000000...		_controlfp	MSVCRT

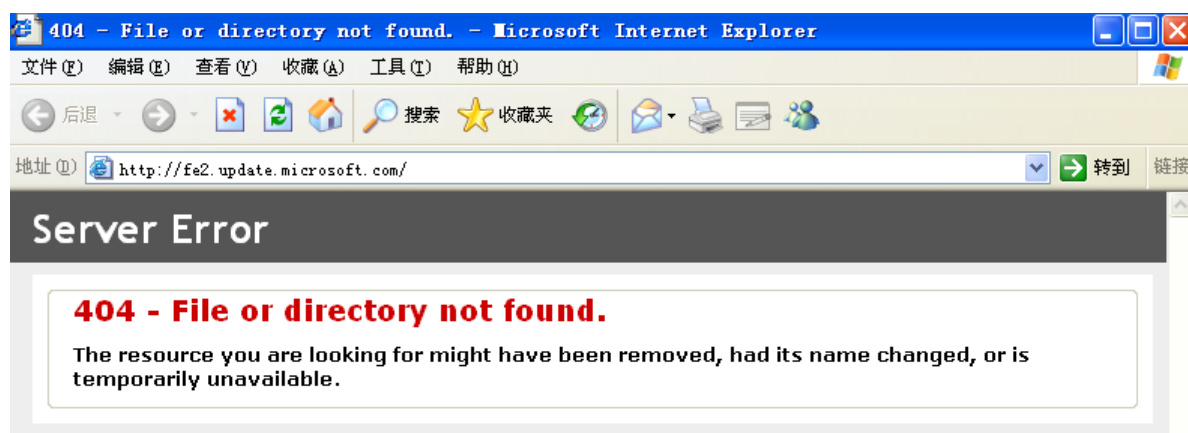
看到包含导入函数CreateRemoteThread。同时，也看到有资源操作函数的导入，例如LoadResource和FindResourceA等。

用Resource Hacker检查恶意代码，可以看到一个名为BIN的程序存储在资源段中：

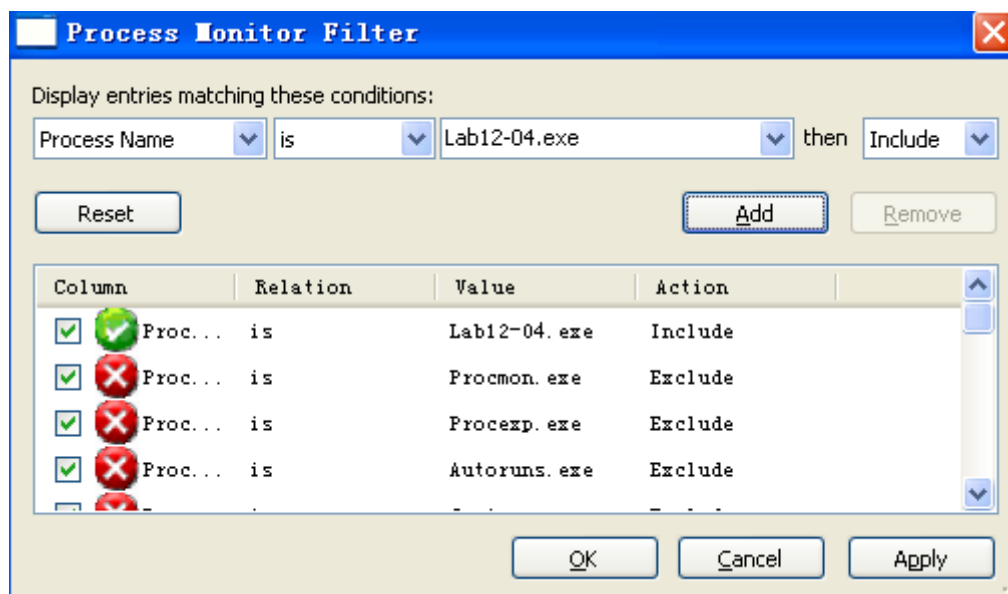


使用 Resource Hacker 查看资源，可以看到 MZ、PE 等信息，确定为一个 PE 文件。

运行恶意代码，可以看到弹出一个explorer程序：



使用Process Monitor工具监视恶意代码，设置过滤器：



Lab12-04.exe	3432	CreateFile	C:\Documents and Settings\Administrator\Local Settings\Temp
Lab12-04.exe	3432	SetRename...	C:\WINDOWS\system32\wupdmgr.exe
Lab12-04.exe	3432	CreateFile	C:\Documents and Settings\Administrator\Local Settings\Temp
Lab12-04.exe	3432	CloseFile	C:\Documents and Settings\Administrator\Local Settings\Temp
Lab12-04.exe	3432	CloseFile	C:\Documents and Settings\Administrator\Local Settings\Temp
Lab12-04.exe	3432	CloseFile	C:\Documents and Settings\Administrator\Local Settings\Temp\winup.exe
Lab12-04.exe	3432	RegOpenKey	HKCU
Lab12-04.exe	3432	RegOpenKey	HKCU\Software\Policies\Microsoft\Control Panel\Desktop
Lab12-04.exe	3432	RegOpenKey	HKCU\Control Panel\Desktop
Lab12-04.exe	3432	RegQueryValue	HKCU\Control Panel\Desktop\MultiUILanguageId
Lab12-04.exe	3432	RegCloseKey	HKCU\Control Panel\Desktop
Lab12-04.exe	3432	RegCloseKey	HKCU
Lab12-04.exe	3432	CreateFile	C:\WINDOWS\system32\wupdmgr.exe
Lab12-04.exe	3432	CreateFile	C:\WINDOWS\system32
Lab12-04.exe	3432	CloseFile	C:\WINDOWS\system32
Lab12-04.exe	3432	WriteFile	C:\WINDOWS\system32\wupdmgr.exe
Lab12-04.exe	3432	ReadFile	C:\Documents and Settings\Administrator\桌面\上机实验样本\Chapter_12L\Lab12-04.exe
Lab12-04.exe	3432	CloseFile	C:\WINDOWS\system32\wupdmgr.exe

Procmon工具告诉我们恶意代码创建一个文件(\TEMP\winup.exe)，并且覆盖了位于 `WINDOWS\System32\wupdmgr.exe` 的Windows更新二进制文件。

比较恶意代码释放的wupdmgr.exe和恶意代码资源节中BIN文件，可以看到它们是相同的(Windows文件保护机制本应该能够恢复原始文件，但是事实上却没有)。

运行netcat工具，可以发现恶意代码试图从 www.practicalmalwareanalysis.com 中下载updater.exe。

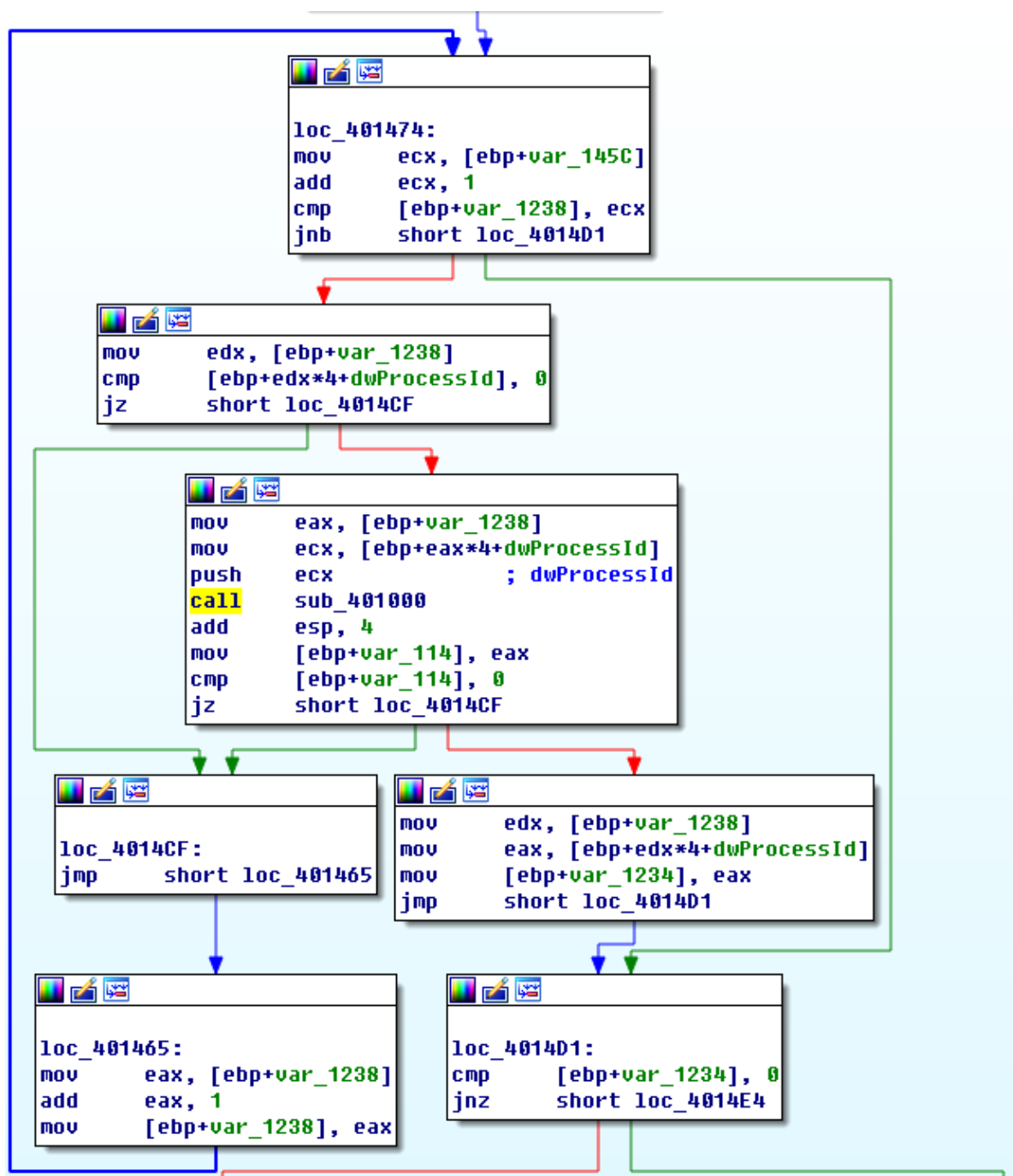
下面使用IDA进行分析。可以看到其通过LoadLibraryA和GetProcAddress解析三个函数，并将三个函数指针分别保存在dword_40312c。

<pre> .text:00401396 .text:004013A0 .text:004013AA .text:004013AF .text:004013B4 .text:004013B8 .text:004013BB .text:004013C1 .text:004013C6 .text:004013CB .text:004013D0 .text:004013D6 .text:004013D7 .text:004013DD .text:004013E2 .text:004013E7 .text:004013EC .text:004013F2 .text:004013F3 .text:004013F9 .text:004013FE .text:00401405 .text:00401407 .text:0040140E .text:00401410 .text:00401417 </pre>	<pre> mov [ebp+var_1234], 0 mov [ebp+var_122C], 0 push offset ProcName ; "EnumProcessModules" push offset aPsapi_dll ; "psapi.dll" call ds:LoadLibraryA push eax ; hModule call ds:GetProcAddress mov dword_40312C, eax push offset aGetmodulebasen ; "GetModuleBaseNameA" push offset aPsapi_dll_0 ; "psapi.dll" call ds:LoadLibraryA push eax ; hModule call ds:GetProcAddress mov dword_403128, eax push offset aEnumprocesses ; "EnumProcesses" push offset aPsapi_dll_1 ; "psapi.dll" call ds:LoadLibraryA push eax ; hModule call ds:GetProcAddress mov dword_403124, eax cmp dword_403124, 0 jz short loc_401419 cmp dword_403128, 0 jz short loc_401419 cmp dword_40312C, 0 jnz short loc_401423 </pre>
--	---

继续分析，可以看到其使用 `muEnumProcess` 列出当前进程，返回 PID，保存到 `dwProcessID` 中

<pre> .text:00401423 loc_401423: .text:00401423 .text:00401429 .text:0040142A .text:0040142F .text:00401435 .text:00401436 .text:0040143C .text:0040143E .text:00401440 .text:00401445 </pre>	<pre> ; CODE XREF: _main+C77j lea eax, [ebp+var_1228] push eax push 1000h lea ecx, [ebp+dwProcessId] push ecx call dword_403124 test eax, eax jnz short loc_40144A mov eax, 1 imp loc_401598 </pre>
---	--

接着为一个循环结构，遍历 PID，将其作为参数传递给 `sub_401000`。步入分析，可以看到 `Str1` 和 `Str2` 两个字符串。接着调用函数，将其返回值进行比较。



下面分析 sub_401174 函数，可以看到其调用 sub_4010FC

```

.text:00401474 loc_401474:                                ; CODE XREF: _main+113↑j
.text:00401474      mov     ecx, [ebp+var_145C]
.text:0040147A      add     ecx, 1
.text:0040147D      cmp     [ebp+var_1238], ecx
.text:00401483      jnb     short loc_4014D1
.text:00401485      mov     edx, [ebp+var_1238]
.text:00401488      cmp     [ebp+edx*4+dwProcessId], 0
.text:00401493      jz      short loc_4014CF
.text:00401495      mov     eax, [ebp+var_1238]
.text:00401498      mov     ecx, [ebp+eax*4+dwProcessId]
.text:004014A2      push    ecx                ; dwProcessId
.text:004014A3      call    sub_401000
.text:004014A8      add     esp, 4
.text:004014AB      mov     [ebp+var_114], eax
.text:004014B1      cmp     [ebp+var_114], 0
.text:004014B8      jz      short loc_4014CF
.text:004014BA      mov     edx, [ebp+var_1238]
.text:004014C0      mov     eax, [ebp+edx*4+dwProcessId]
.text:004014C7      mov     [ebp+var_1234], eax
.text:004014CD      jmp     short loc_4014D1

```

继续分析 sub_401120，可以看到其调用 lookupPrivilegeValueA 函数用于提升权限：

```

.text:00401120 loc_401120:                                ; CODE XREF: sub_4010FC+1B7j
.text:00401120      mov     [ebp+NewState.PrivilegeCount], 1
.text:00401127      mov     [ebp+NewState.Privileges.Attributes], 2
.text:0040112E      lea     ecx, [ebp+NewState.Privileges]
.text:00401131      push    ecx                ; lpLuid
.text:00401132      mov     edx, [ebp+lpName]
.text:00401135      push    edx                ; lpName
.text:00401136      push    0                  ; lpSystemName
.text:00401138      call    ds:LookupPrivilegeValueA
.text:0040113E      test    eax, eax
.text:00401140      jnz     short loc_401153
.text:00401142      mov     eax, [ebp+TokenHandle]
.text:00401145      push    eax                ; hObject
.text:00401146      call    ds:CloseHandle
.text:0040114C      mov     eax, 1
.text:00401151      jmp     short loc_40116E

```

继续分析 loc_4011A1:

```

.text:004011A1 loc_4011A1:                                ; CODE XREF: sub_401174+277j
.text:004011A1      push    2                  ; lpProcName
.text:004011A3      push    offset LibFileName ; "sfc_os.dll"
.text:004011A8      call    ds:LoadLibraryA
.text:004011AE      push    eax                ; hModule
.text:004011AF      call    ds:GetProcAddress
.text:004011B5      mov     lpStartAddress, eax
.text:004011BA      mov     eax, [ebp+dwProcessId]
.text:004011BD      push    eax                ; dwProcessId
.text:004011BE      push    0                  ; bInheritHandle
.text:004011C0      push    1FFFFFFh          ; dwDesiredAccess
.text:004011C5      call    ds:OpenProcess
.text:004011CB      mov     [ebp+hProcess], eax
.text:004011CE      cmp     [ebp+hProcess], 0
.text:004011D2      jnz     short loc_4011D8
.text:004011D4      xor     eax, eax
.text:004011D6      jmp     short loc_4011F8

```

可以看到其调用 LoadLibraryA 函数用于装载 sfc_os.dll，然后借助 GetProcAddress 函数获取其中编号为 2 的函数地址保存到 lpStartAddress 中，调用 OpenProcess 打开 winLogon.exe，将其句柄保存到 hProcess。

接着继续分析 loc_4011D8:

```

.text:004011D8 loc_4011D8:                                ; CODE XREF: sub_401174+5E7j
.text:004011D8      push    0                  ; lpThreadId
.text:004011DA      push    0                  ; dwCreationFlags
.text:004011DC      push    0                  ; lpParameter
.text:004011DE      mov     ecx, lpStartAddress
.text:004011E4      push    ecx                ; lpStartAddress
.text:004011E5      push    0                  ; dwStackSize
.text:004011E7      push    0                  ; lpThreadAttributes
.text:004011E9      mov     edx, [ebp+hProcess]
.text:004011EC      push    edx                ; hProcess
.text:004011ED      call    ds:CreateRemoteThread
.text:004011F3      mov     eax, 1

```

调用 CreateRemoteThread 函数，其 hProcess 参数是 winlogon.exe 的句柄。

004011de 处的 lpStartAddress 是 sfc_os.dll 中序号为 2 的函数的指针，负责向 winlogon.exe 注入一个线程，而该线程就是 sfc_os.dll 的序号为 2 的函数。

接着调用有关资源节的函数，写入到 C:\windows\system32\wupdmgr.exe :

```

.text:00401273      call     ds:GetWindowsDirectoryA
.text:00401279      push    offset aSystem32Wupdmgr ; "\\system32\\wupdmgr.exe"
.text:0040127E      lea     ecx, [ebp+Buffer]
.text:00401284      push    ecx
.text:00401285      push    offset Format           ; "%5S"
.text:0040128A      push    10Eh                   ; Count
.text:0040128F      lea     edx, [ebp+Dest]
.text:00401295      push    edx                     ; Dest
.text:00401296      call    ds:_snprintf
.text:0040129C      add     esp, 14h
.text:0040129F      push    0                      ; lpModuleName
.text:004012A1      call    ds:GetModuleHandleA
.text:004012A7      mov     [ebp+hModule], eax
.text:004012AA      push    offset Type             ; "BIN"
.text:004012AF      push    offset Name             ; "#101"
.text:004012B4      mov     eax, [ebp+hModule]
.text:004012B7      push    eax                     ; hModule
.text:004012B8      call    ds:FindResourceA
.text:004012BE      mov     [ebp+hResInfo], eax
.text:004012C4      mov     ecx, [ebp+hResInfo]
.text:004012CA      push    ecx                     ; hResInfo
.text:004012CB      mov     edx, [ebp+hModule]
.text:004012CE      push    edx                     ; hModule
.text:004012CF      call    ds:LoadResource
.text:004012D5      mov     [ebp+lpBuffer], eax
.text:004012D8      mov     eax, [ebp+hResInfo]
.text:004012DE      push    eax                     ; hResInfo
.text:004012DF      mov     ecx, [ebp+hModule]

```

继续分析，可以看到恶意代码通过 WinExec 来启用已经被改写过的 wupdmgr.exe：

```

.text:00401321      push    edx                     ; lpBuffer
.text:00401322      mov     eax, [ebp+hFile]
.text:00401328      push    eax                     ; hFile
.text:00401329      call    ds:WriteFile
.text:0040132F      mov     ecx, [ebp+hFile]
.text:00401335      push    ecx                     ; hObject
.text:00401336      call    ds:CloseHandle
.text:0040133C      push    0                      ; uCmdShow
.text:0040133E      lea     edx, [ebp+Dest]
.text:00401344      push    edx                     ; lpCmdLine
.text:00401345      call    ds:WinExec
.text:0040134B      pop     edi
.text:0040134C      mov     esp, ebp
.text:0040134E      pop     ebp
.text:0040134F      retn
.text:0040134F      sub_4011FC endp

```

在 0040133c 处可以看到 `push 0` 指令，将 0 作为 `uCmdShow` 参数值来启动，从而实现隐藏程序窗口的目的。

2. 问题解答

(1) 位置 0x401000 的代码完成了什么功能？

恶意代码查看给定PID是否为winlogon.exe 进程。

(2) 代码注入了哪个进程？

注入了 winlogon.exe 进程。

(3) 使用 LoadLibraryA 装载了哪个 DLL 程序？

装载 sfc_os.dll 文件，禁用 Windows 的文件保护机制。

(4) 传递给 CreateRemoteThread 调用的第 4 个参数是什么？

传递给 CreateRemoteThread 的第 4 个参数是一个函数指针，指向 sfcos.dll 中一个未命名的序号为 2 的函数 (SfcTerminateWatcherThread)

(5) 二进制主程序释放出了哪个恶意代码？

恶意代码从资源段中释放一个二进制文件，并且将这个二进制文件覆盖旧的 Windows 更新程序 (wupdmgr.exe)。覆盖真实的 wupdmgr.exe 之前，恶意代码将它复制到 %TEMP% 目录，供以后使用。

(6) 释放出恶意代码的目的是什么？

恶意代码向 winlogon.exe 注入一个远程线程，并且调用 sfcos.dll 的一个导出函数 (序号为 2 的 SfcTerminateWatcherThread)，在下次启动之前禁用 Windows 的文件保护机制。恶意代码通过用这个二进制文件来更新自己的恶意代码，并且调用原始的二进制文件 (位于 %TEMP% 目录) 来特洛伊木马化 wupdmgr.exe 文件。

三、yara 规则

基于以上分析编写 yara 规则如下：

```
rule lab1201exe{
  strings:
    $dll1 = "Lab12-01.dll"
    $string1 = "GetModuleBaseNameA"
    $dll2 = "psapi.dll"
    $string2 = "EnumProcessModules"
  condition:
    filesize < 200KB and uint16(0) == 0x5A4D and uint16(uint16(0x3C)) ==
    0x00004550 and all of them
}
rule lab1202exe{
  strings:
    $string1 =
    "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
    AAAAA"
    $exe1 = "svchost.exe"
  condition:
    filesize < 200KB and uint16(0) == 0x5A4D and uint16(uint16(0x3C)) ==
    0x00004550 and all of them
}
rule lab1203exe{
  strings:
    $log = "practicalmalwareanalysis.log"
    $func = "VirtualAlloc"
    $string1 = "TerminateProcess"
    $string2 = "\r\n[window:"
  condition:
    filesize < 200KB and uint16(0) == 0x5A4D and uint16(uint16(0x3C)) ==
    0x00004550 and all of them
}
rule lab1204exe{
  strings:
```



```

$string1 = "www.practicalmalwareanalysis.com"
$exe1 = "wupdmgr.exe"
$exe2 = "winlogon.exe"

condition:
    filesize < 200KB and uint16(0) == 0x5A4D and uint16(uint16(0x3C)) ==
0x00004550 and all of them
}
rule lab1201dll{
strings:
    $string1 = "Practical Malware Analysis %d"
    $string2 = "Press OK to reboot"
condition:
    filesize < 200KB and uint16(0) == 0x5A4D and uint16(uint16(0x3C)) ==
0x00004550 and all of them
}

```

运行结果如下，可见所有恶意文件都已被检出：

```

PS D:\大三上\大三上\恶意代码分析与防治技术\Practical Malware Analysis Labs\Practical Malware Analysis Labs\BinaryCollection\Chapter_1
2L> .\yara64.exe -r -c .\lab12.yar .
.\lab12.yar: 0
.\lab12-01.dll: 1
.\lab12-01.exe: 1
.\lab12-04.exe: 1
.\lab12-03.exe: 1
.\lab12-02.exe: 1

```

四、IDA Python脚本编写

遍历所有函数，排除库函数或简单跳转函数，当反汇编的助记符为call或者jmp且操作数为寄存器类型时，输出该行反汇编指令。

定位当前函数：

- 获取光标所在函数的名称、起止地址。

检查函数属性：

- 打印函数是否具有特定标志位（如 `FUNC_NORET`、`FUNC_FRAME` 等）。

分析函数内部指令：

- 如果函数不是库函数或跳板函数，则列出函数内的所有 `call` 和 `jmp` 指令的地址和反汇编内容。

```

import idutils
ea=idc.ScreenEA()
funcName=idc.GetFunctionName(ea)
func=idapi.get_func(ea)
print("FuncName:%s"%funcName)
print "Start:0x%x,End:0x%x" % (func.startEA,func.endEA)

flags = idc.GetFunctionFlags(ea)
if flags&FUNC_NORET:
    print "FUNC_NORET"
if flags & FUNC_FAR:
    print "FUNC_FAR"
if flags & FUNC_STATIC:
    print "FUNC_STATIC"
if flags & FUNC_FRAME:
    print "FUNC_FRAME"

```

```

if flags & FUNC_USERFAR:
    print "FUNC_USERFAR"
if flags & FUNC_HIDDEN:
    print "FUNC_HIDDEN"
if flags & FUNC_THUNK:
    print "FUNC_THUNK"
if not(flags & FUNC_LIB or flags & FUNC_THUNK):# 获取当前函数中call或者jmp的指令
    dism_addr = list(idautils.FuncItems(ea))
    for line in dism_addr:
        m = idc.GetMnem(line)
        if m == "call" or m == "jmp":
            print "0x%x %s" % (line,idc.GetDisasm(line))

```

在Lab11-03.dll的IDA Pro中运行该脚本

分析出光标所在位置的函数StartAddress的call指令和jmp指令：

```

FuncName:sub_401000
Start:0x401000,End:0x4010fc
FUNC_FRAME
0x401078 call    ds:OpenProcess
0x40109b call    dword_40312C
0x4010bc call    dword_403128
0x4010cd call    ds:_stricmp
0x4010de call    ds:CloseHandle
0x4010e9 jmp     short loc_4010F7
0x4010ef call    ds:CloseHandle

```

五、实验结论及心得体会

1. 在Lab12-02的恶意代码分析中发现了一种新的隐蔽执行恶意代码的方式，使用**进程替换**，并且恶意代码的有效载荷被**XOR加密**后保存在这个程序的资源节。同时在分析过程中也使用了新的分析工具：Resource Hacker 和 WinHex。

Resource Hacker 是一款功能强大的 Windows 资源编辑工具，广泛用于查看、修改、提取和替换 Windows 可执行文件（如 .exe、.dll、.sys 等）中的资源。

WinHex 是一款功能强大的十六进制编辑器和数据恢复工具，广泛用于数字取证、数据恢复、硬盘分析、系统调试和磁盘管理等领域。它能够查看、编辑和处理各种文件格式以及存储设备的底层数据，包括文件、磁盘、内存等。

2. 在Lab12-04的实验中分析了恶意代码是通过禁用Windows文件保护机制来修改Windows功能的一种通用方法。在本实验中，恶意代码特洛伊木马化Windows的更新进程，并且创建了它自己的更新程序。因为恶意代码没有完全破坏原始的Windows更新二进制文件，所以感染这个恶意代码机器的用户仍会看到正常的Windows更新功能。